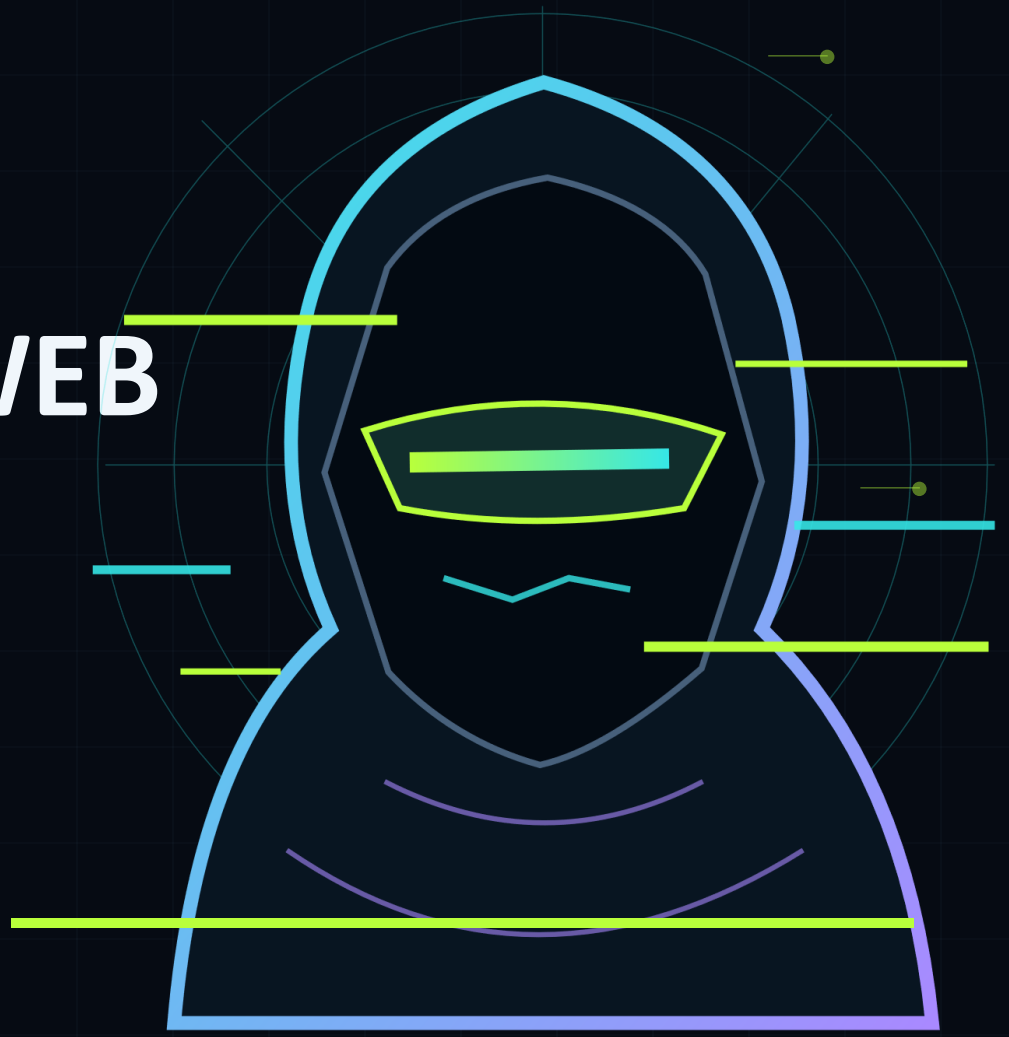
vlab

INFOESTE // CYBER LAB 2026

COMO HACKERS QUEBRAM APLICAÇÕES WEB



ATAQUE & DEFESA NA PRÁTICA

WINDOWS 11 • BURP • ZAP • CTF

8 HORAS / 201 SLIDES

QUEM FALA COM VOCÊS

Henrique Otsuka

Head de Tecnologia da Informação · VLab Health



FORMAÇÃO & ESPECIALIZAÇÃO

- **Graduado em Redes de Computadores**
- **Especialista em Ethical Hacking & Cybersecurity**
- **Especialista em Desenvolvimento Avançado com Golang**

Também atuo com: Go · Python · React · Flutter · streaming HLS/RTMP/WebRTC · sistemas distribuídos · AWS · OCI

QUEM FALA COM VOCÊS

Marcos Santos

Desenvolvedor Fullstack · VLab Health



FORMAÇÃO & ESPECIALIZAÇÃO

- **Graduado em Análise e Desenvolvimento de Sistemas**

Também atua com: Go · Python · React

vlab

VLab Health

Infraestrutura de cuidado materno-infantil · startup, 8 anos

O QUE FAZEMOS

- Conectamos clínicas, hospitais, marcas e gestantes & mães
- App de acompanhamento para gestantes e mães
- Integração de exames de diagnóstico por imagem
- Dados de saúde: segurança e compliance como requisito

QUEM ATENDEMOS

- Os maiores laboratórios de diagnóstico por imagem
- Clínicas e hospitais
- Marcas do ecossistema materno-infantil
- Gestantes e mães, direto pelo app

8 anos

de operação

+ 2 milhões

de exames realizados

~ 300 mil

mães no app

vlab

WELCOME / MISSION

VAI INTERCEPTAR. TESTAR. QUEBRAR. CORRIGIR.

8 horas de laboratório, do HTTP ao mini CTF.

vlab

CONTEXT / THREAT LANDSCAPE

01000011
01011001
01000010
01000101
01010010

00

POR QUE ISSO IMPORTA

Antes de atacar, entenda o campo de batalha: escala, custo e anatomia de um ataque.

vlab

O cibercrime em escala

US$ 10,5 tri

em perdas globais por ano (2025) — seria a 3ª seria a 3ª maior economia do mundo

+15% / ano

crescimento projetado → US$ 12 tri até 2031 (era US$ 3 tri em 2015)

a cada 11s

uma empresa é atingida por ransomware; ransomware; dano de ransomware → US$ 265 US$ 265 bi/ano até 2031

A LEITURA

Não é “se”, é “quando”. A superfície digital cresce mais rápido que a defesa, e a probabilidade de detecção e punição do cibercrime é estimada em cibercrime é estimada em menos de 1%. Atacar ficou barato e escalável; defender virou requisito de negócio.

Fonte: Cybersecurity Ventures — Cybersecurity Almanac 2025

vlab

Quanto custa uma violação

US$ 4,44 mi

custo médio global de uma violação de dados dados em 2025

US$ 10,22 mi

média nos EUA — recorde histórico

US$ 7,42 mi

setor de saúde: o mais caro, ano após ano ano

- **241 dias** — tempo médio para identificar + conter uma violação
- **Phishing (16%)** — vetor inicial mais comum — ultrapassou credenciais roubadas

Fonte: IBM — Cost of a Data Breach Report 2025

vlab

Quando acontece de verdade

Ataque ao Pix (C&M Software) — 2025

~R$ 800 milhões desviados em horas. Um funcionário vendeu login e senha — maior golpe ao sistema financeiro do Brasil.

JBS — 2021

US$ 11 milhões de resgate em bitcoin (ransomware REvil); fábricas paradas paradas nos EUA, Canadá e Austrália.

Mercado Livre — 2022

~300 mil usuários expostos após invasão do código-fonte; autoria atribuída ao atribuída ao grupo Lapsus$.

Lapsus$ no Brasil — 2021–22

O mesmo grupo (com integrantes brasileiros) atingiu Ministério da Saúde Saúde (ConecteSUS), Localiza, Claro, Nvidia e Samsung.

Repare: quase todos começaram com **credencial roubada ou engenharia social** — não com um exploit sofisticado. É exatamente o que vocês vão o que vocês vão explorar e defender hoje.

Fontes: Banco Central / Polícia Civil-SP · Wall Street Journal · comunicados das empresas (2021–2025)

vlab

Os pilares da segurança

C

Confidencialidade

só quem pode ver, vê

→ **Cifra, controle de acesso**

I

Integridade

o dado não foi adulterado

→ **Hash, assinatura, validação**

A

Disponibilidade

o sistema responde quando precisa

→ **Redundância, rate limit, backup**

A tríade CIA. Complementada por Autenticidade (quem é de fato) e Não-repúdio (não dá pra negar o que fez). Toda falha do curso quebra pelo curso quebra pelo menos um desses pilares.

vlab

O fim do perímetro

CASTELO E FOSSO

O modelo antigo: muralha forte na borda, confiança total dentro. Quem passou do firewall era “confiável”.

POR QUE RUIU

- Nuvem: o recurso já está “fora”
- SaaS e APIs: sem borda clara
- Mobile e home office
- Ameaça interna e credencial roubada

ZERO TRUST

“Nunca confie, sempre verifique.” Cada requisição é autenticada e autorizada, venha de onde vier. A fronteira de confiança deixou de ser a rede e deixou de ser a rede e passou a ser cada endpoint — exatamente o que o IDOR e o JWT do lab exploram.

vlab

O ciclo de vida de um ataque

Cyber Kill Chain (Lockheed Martin) — as fases que um atacante percorre:

01	02	03	04	05	06	07
Reconhecimento	**Armamento**	**Entrega**	**Exploração**	**Instalação**	**C2**	**Ação**
mapear o alvo, achar entradas	preparar o payload/exploit	fazer o payload chegar chegar	disparar a vulnerabilidade vulnerabilidade	persistência no alvo	canal de comando e controle	exfiltrar, cifrar, destruir destruir

No lab vocês vivem o início da cadeia: Reconhecimento (RECON, robots.txt → /diag) e Exploração (IDOR, SQLi, XSS, JWT). Interromper qualquer fase quebra o qualquer fase quebra o ataque — defesa em profundidade.

vlab

OBJETIVOS

O que você vai conseguir fazer

01

Ler uma request

Entender o que o browser realmente envia.

02

Manipular uma request

Alterar parâmetros com Burp Repeater.

03

Encontrar falhas

Reconhecer IDOR, SQLi, XSS e falhas de auth.

04

Corrigir a causa

Aplicar controles no servidor e no código.

REQUEST PIPELINE

O ciclo que vamos repetir

Pensar como atacante, trabalhar como desenvolvedor.

01 OBSERVAR › 02 HIPÓTESE › 03 TESTAR › 04 EVIDÊNCIA › 05 CORRIGIR

vlab

8H / RUN OF SHOW

Mapa das 8 horas

00:00	Abertura e setup	30 min
00:30	HTTP + DevTools	45 min
01:15	Burp Suite	50 min
02:05	IDOR	50 min
02:55	SQL Injection	50 min
03:45	XSS	50 min
04:35	Autenticação + JWT	45 min
05:20	ZAP + análise	35 min
05:55	Defesa: modo seguro + Cloud	50 min
06:45	Mini CTF e encerramento	75 min

SAFETY / SCOPE

SÓ TESTE O QUE VOCÊ TEM AUTORIZAÇÃO PARA TESTAR. TESTAR.

Alvo do curso: aplicação local e propositalmente vulnerável. Nada de testar sistemas da sistemas da faculdade, redes externas ou contas de terceiros.

CONTRASTE

Ataque e defesa: duas lentes

MENTALIDADE DE ATAQUE

- Qual entrada eu controlo?
- Que regra o servidor esqueceu?
- Como provar o impacto?

MENTALIDADE DE DEFESA

- Onde está a fronteira de confiança?
- Qual controle deve existir?
- Como garantir com teste?

vlab

PROVOCAÇÃO
O

“Mas no frontend não aparece esse botão...”

E desde quando esconder um botão virou controle de acesso?
acesso?

ATTACK SURFACE

Ferramentas do nosso arsenal

01

Browser + DevTools

Network, cookies, storage e DOM.

02

Burp Community

Proxy, histórico e Repeater.

03

Docker Desktop

O laboratório roda localmente.

04

ZAP

Scanning controlado e triagem.

vlab

DEFENSE CHECKLIST

Pré-flight no Windows 11

Não vamos instalar WSL nem transferir ZIP. O código vem por git clone.

- ☑ Windows 11 e WSL 2 já preparados pelo laboratório
- ☑ Docker Desktop aberto (Linux engine ativo)
- ☑ Git disponível no PowerShell
- ☑ Chrome / Edge operacional
- ☑ Burp Suite Community disponível
- ☑ OWASP ZAP disponível
- ☑ Editor VS Code / Zed instalado
- ☑ Internet e GitHub acessíveis

vlab

WELCOME / MISSION

AMBIENTE PRONTO. AGORA É CÓDIGO.

TERMINAL / POWERSHELL

01 — Validar Docker e Git

SESSION // LOCAL ONLY

```
git --version
docker --version
docker compose version
docker info --format "{{.ServerVersion}}"
```

Quatro comandos no PowerShell. O último precisa retornar a versão do Docker Engine.

vlab

TERMINAL / POWERSHELL

02 — Clonar o projeto com Git

SESSION // LOCAL ONLY

```
cd $HOME
git clone https://github.com/HenriqueOtsuka/neon-shop-infoeste.git
cd neon-shop-infoeste
```

vlab

TERMINAL / POWERSHELL

03 — Conferir a pasta clonada

SESSION // LOCAL ONLY

```
pwd
git status
Test-Path .\compose.yaml
Get-ChildItem -Name
```

Esperado: git status limpo e Test-Path retornando True. Execute Docker nessa pasta.

vlab

TERMINAL / POWERSHELL

04 — Construir e iniciar o laboratório

SESSION // LOCAL ONLY

```
docker compose up -d --build
```

O comando constrói a imagem local e inicia o container. Pode levar mais tempo no primeiro uso.

vlab

TERMINAL / POWERSHELL

05 — Conferir os serviços do Compose

SESSION // LOCAL ONLY

```
docker compose ps
docker ps --filter name=infoeste-neon-shop
```

Esperado: serviço neon-shop em execução, com porta 127.0.0.1:8088 e estado healthy.

TERMINAL / POWERSHELL

06 — Testar a API via curl no Windows

SESSION // LOCAL ONLY

```
curl.exe -i http://127.0.0.1:8088/api/health
```

Esperado: HTTP/1.0 200 OK e JSON com status: ok, mode: vulnerable.

TERMINAL / POWERSHELL

07 — Teste alternativo PowerShell

SESSION // LOCAL ONLY

```
Invoke-RestMethod http://localhost:8088/api/health
Test-NetConnection 127.0.0.1 -Port 8088
```

Esperado: status = ok, lab = neon-shop, mode = vulnerable; TcpTestSucceeded = True.

vlab

STEP BY STEP / FOLLOW ALONG

08 — Como saber que está tudo OK?

GIT	Projeto baixado	git status executa sem erro.
COMPOSE	Container ativo	docker compose ps: Up (healthy).
HTTP	Endpoint responde	curl.exe .../api/health → 200.
API	Estado correto	status: ok / mode: vulnerable.
BROWSER	Interface abre	http://localhost:8088/login

Se um teste falhar, resolva antes de começar as explorações.

vlab

TERMINAL / POWERSHELL

09 — Abrir a aplicação no navegador

SESSION // LOCAL ONLY

```
Start-Process "http://localhost:8088/login"
Start-Process "http://localhost:8088/api/health"
```

Login: aluno@lab.local / exemplo123. Não use HTTPS; este lab utiliza HTTP local.

vlab

EXECUTE / PASSO A PASSO

Primeiro login — siga na tela

01	**Abra o endereço**	No Chrome/Edge: http://localhost:8088/login
02	**Preencha e-mail**	aluno@lab.local (conta fictícia A)
03	**Preencha senha**	exemplo123 e clique AUTENTICAR
04	**Confirme a tela**	Você será enviado para /orders.

OK **Tela de pedidos aparece com a conta Alex autenticada.**

SE FALHAR: Se retornar 401, reveja e-mail/senha; se não abrir, use o health check.

EXECUTE / PASSO A PASSO

Qual programa você deve abrir?

01	**Docker Desktop**	Apenas para garantir que o servidor está executando.
02	**PowerShell**	Para git, Docker, curl.exe e diagnóstico.
03	**Chrome ou Edge**	Para abrir a loja e usar o DevTools.
04	**Burp Suite**	Depois, para interceptar e editar HTTP.

OK **Cada ferramenta tem função diferente; não instale Python nem SQLite.**

SE FALHAR: Se algum programa estiver ausente, peça auxílio à equipe técnica da universidade.

TERMINAL / POWERSHELL

10 — Logs: descobrir por que falhou

SESSION // LOCAL ONLY

```
docker compose logs --tail=80 neon-shop
docker compose logs -f neon-shop
```

O segundo comando acompanha logs em tempo real. Pressione Ctrl+C para sair dos logs.

TERMINAL / POWERSHELL

11 — Reiniciar sem perder progresso

SESSION // LOCAL ONLY

```
docker compose restart neon-shop
docker compose ps
curl.exe http://127.0.0.1:8088/api/health
```

restart reinicia o container. Não apaga o volume de dados nem as flags.

TERMINAL / POWERSHELL

12 — Atualizar o projeto por Git

SESSION // LOCAL ONLY

```
git status
git pull --ff-only origin main
docker compose up -d --build
```

Execute somente se o instrutor avisar que houve atualização no repositório.

vlab

TERMINAL / POWERSHELL

13 — Parar sem apagar dados

SESSION // LOCAL ONLY

```
docker compose down

# Para voltar depois:
docker compose up -d
```

down preserva o volume. Não use docker compose down -v durante a aula.

STEP BY STEP / FOLLOW ALONG

7. Contas fictícias do laboratório

A	Aluno A	aluno@lab.local / exemplo123
B	Aluno B	beatriz@lab.local / exemplo123
ADM	Administrador	admin@lab.local / exemplo123
LINK	Abra /login	http://localhost:8088/login

Use apenas credenciais fornecidas; nunca use e-mail ou senha real.

vlab

STEP BY STEP / FOLLOW ALONG

8. Mapa da NEON SHOP

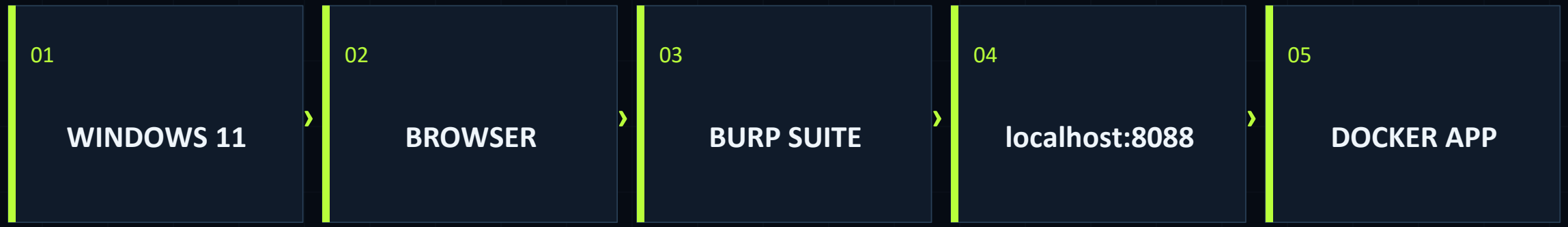
/orders	IDOR	Pedidos dos usuários de teste.
/search	SQL Injection	Busca por produtos e parâmetros.
/comments	XSS	Mural de comentários.
/admin	JWT/Auth	Autorização na API de relatórios.
/ctf	Mini CTF	Cole as flags e acompanhe o placar.

Prefixe as rotas por http://localhost:8088. Nenhum endereço externo faz parte do alvo.

REQUEST PIPELINE

Arquitetura do laboratório

Sem Kali, sem VM adicional e sem dependência de cloud.



vlab

LAB: NEON SHOP

Uma aplicação fictícia

/login

Identidades

Entrar com contas de teste.

/orders

Pedidos

Ver pedidos e perfis.

/search

Busca

Procurar por produtos.

/comments

Comentários

Publicar mensagens.

HANDS-ON / LAB CHALLENGE

MISSÃO 00 — Reconhecimento

08 MIN

NÍVEL 01

Abra a aplicação de laboratório e identifique 3 ações que geram requisições.

- Acesse duas telas.
- Abra Network no DevTools.
- Localize método, URL e status.

Entregável: uma request explicada em voz alta.

EXECUTE / PASSO A PASSO

DevTools — onde fica o Network?

01	**Abra a loja**	No Chrome/Edge normal: http://localhost:8088/login
02	**Abra o DevTools**	Pressione F12 ou Ctrl+Shift+I.
03	**Selecione Network**	Clique na aba Network / Rede, na parte superior.
04	**Ative a gravação**	O círculo de gravação deve estar vermelho.

OK **A aba Network está aberta e preparada para registrar requisições.**

SE FALHAR: Se não aparecer a aba, use » no topo do DevTools e escolha Network.

EXECUTE / PASSO A PASSO

DevTools — capture seu primeiro POST

01	**Limpe o histórico**	No Network, clique em Clear (ícone de limpar).
02	**Marque Fetch/XHR**	Filtre requisições geradas pelo JavaScript.
03	**Faça o login**	Na página /login, clique AUTENTICAR.
04	**Encontre /api/login**	Clique na linha POST /api/login.

OK

A lista contém POST /api/login e status 200.

SE FALHAR: Se não aparecer: desative filtros, ative Preserve log e faça logout/login novamente.

EXECUTE / PASSO A PASSO

DevTools — leia as informações

01	**Headers**	Encontre Request URL, Request Method e Status Code.
02	**Payload**	Observe email e password no corpo JSON.
03	**Response**	Veja user e token devolvidos no JSON.
04	**Não copie senhas reais**	Estas credenciais são apenas do laboratório.

OK **Você distingue dados ENVIADOS dos dados RECEBIDOS.**

SE FALHAR: Se Response estiver vazia, confirme se a request selecionada é /api/login.

KEY TAKEAWAY

ANTES DE QUEBRAR, APRENDA A OBSERVAR.

A investigação começa com evidências — não com payloads aleatórios.

vlab

EXECUTE / PASSO A PASSO

Reconhecimento — pista no robots.txt

01	**Abra uma nova aba**	Digite http://localhost:8088/robots.txt
02	**Leia a resposta**	Procure a linha iniciada por Disallow:.
03	**Copie só o caminho**	/api/internal/diag
04	**Abra no mesmo host**	http://localhost:8088/api/internal/diag

OK **A API devolve um JSON com um campo evidence.**

SE FALHAR: Se obtiver 404: confira se o laboratório está em mode: vulnerable.

vlab

EXECUTE / PASSO A PASSO

Reconhecimento — registrar a primeira flag

01	**Localize evidence**	Na resposta JSON do endpoint de diagnóstico.
02	**Copie o valor**	A flag começa com NEON{RECON_...}.
03	**Abra /ctf**	Entre com seu usuário de laboratório.
04	**Cole e clique VALIDAR**	Veja a pontuação da missão RECON.

OK **Pontuação RECON: 100 pontos no scoreboard.**

SE FALHAR: Se “Flag inválida”, copie todo o valor incluindo NEON{ e } (cada PC tem flags próprias).

vlab

MODULE 01 / ACCESS GRANTED

01

HTTP + BROWSER

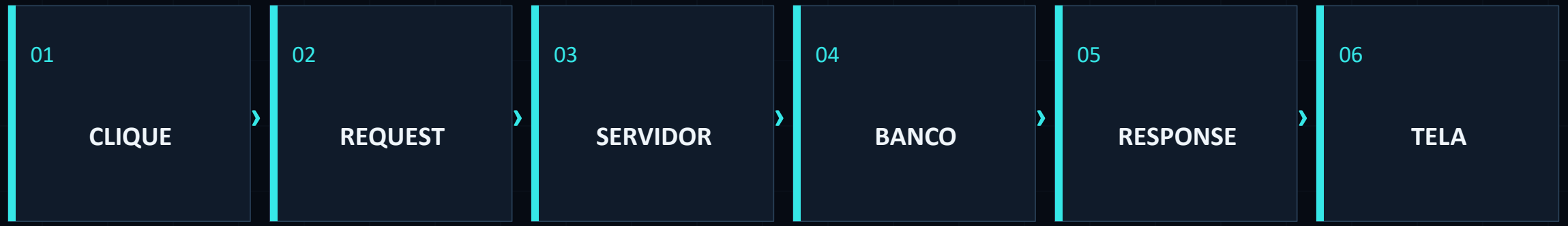
Da tela bonita ao protocolo real.

01101001 10010110 11100010
01011011 00110100 11001010
10101101 01001011 11101001

REQUEST PIPELINE

Clicou em “Entrar”. E agora?

O navegador não é o backend. É apenas o cliente.



RAW HTTP / INSPECT

Anatomia de uma request

REQUEST / RESPONSE

```
POST /api/login HTTP/1.1
Host: localhost:8088
Content-Type: application/json

{"email":"aluno@lab.local",
 "password":"exemplo123"}
```

Método, caminho, headers e body.

RAW HTTP / INSPECT

Anatomia de uma response

REQUEST / RESPONSE

```
HTTP/1.0 200 OK
Content-Type: application/json
Set-Cookie: session=aluno-A; Path=/; HttpOnly

{"user":{"id":"aluno-A","role":"student"},
 "token":"eyJ..."}
```

Exemplo abreviado da resposta real de /api/login: cookie de sessão + JSON com user e JWT.

vlab

ATTACK SURFACE

Os verbos que vamos encontrar

GET

Consultar

Buscar recursos ou listas.

POST

Criar / executar

Enviar dados ou iniciar uma ação.

PUT / PATCH

Alterar

Atualizar um recurso.

DELETE

Remover

Excluir um recurso.

ATTACK SURFACE

HTTP status: sinais do servidor

200

OK

Resposta atendida.

401

Não autenticado

Falta identidade válida.

403

Proibido

Sem permissão para esta ação.

500

Erro interno

Falha do lado do servidor.

VISUAL MODEL

O cookie viaja com a request

Cookie: session=abc123

Identificador de sessão: o servidor decide quem você é.

vlab

CONTRASTE

AUTENTICAÇÃO ≠ AUTORIZAÇÃO

AUTHN // QUEM É VOCÊ?

- Login
- Sessão / token
- Identidade validada

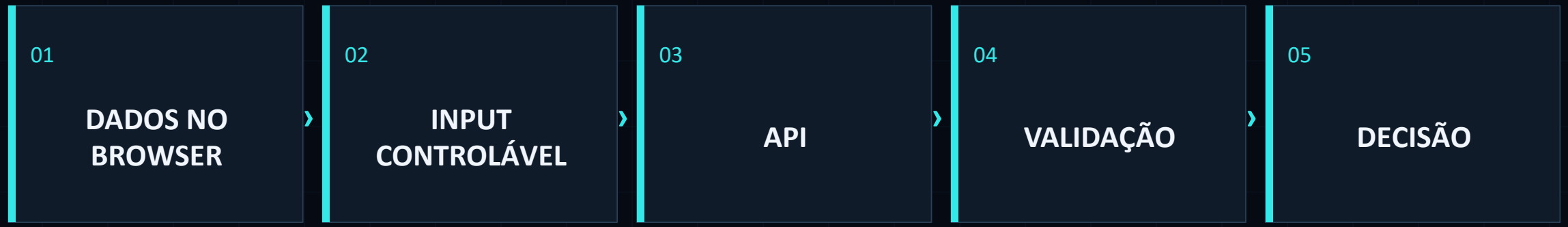
AUTHZ // PODE FAZER ISSO?

- Dono do recurso
- Permissão por ação
- Escopo de organização

REQUEST PIPELINE

Quem está confiando em quem?

Tudo vindo do cliente deve ser tratado como não confiável.



vlab

HANDS-ON / LAB CHALLENGE

MISSÃO 01 — O Network denuncia

10 MIN

NÍVEL 01

Descubra uma chamada HTTP que o frontend faz “escondida” do usuário.

- Filtre Fetch/XHR no DevTools.
- Abra Request Headers e Payload.
- Compare com a aba Response.

Entregável: URL, método, request e status.

ATTACK SURFACE

DevTools: onde clicar

NETWORK

Tráfego

Fetch/XHR, filtros e detalhes.

APPLICATION

Sessões

Cookies, localStorage, sessionStorage.

ELEMENTS

DOM

HTML renderizado e eventos.

CONSOLE

Diagnóstico

Erros JavaScript e logs.

vlab

EXECUTE / PASSO A PASSO

DevTools — cookies e identidade

01	**Com login feito**	Abra F12 → Application (ou Aplicativo).
02	**Abra Storage / Cookies**	Selecione http://localhost:8088.
03	**Procure session**	É o cookie enviado nas requisições autenticadas.
04	**Volte ao Network**	Clique GET /api/me e examine Request Headers.

OK **A request autenticada inclui Cookie: session=...**

SE FALHAR: Se não houver cookie: refaça login; não tente ler cookie HttpOnly por document.cookie.

vlab

EXECUTE / PASSO A PASSO

DevTools — filtre a URL sem confundir a tela

01	**Abra /orders**	Veja seus pedidos no navegador normal.
02	**Acesse Network**	Marque Fetch/XHR no DevTools.
03	**Digite /api/orders**	No campo de filtro do Network.
04	**Clique CONSULTAR**	Veja a nova linha GET /api/orders/1001.

OK **A URL exata identifica o recurso consultado.**

SE FALHAR: Se não encontrar, limpe o filtro e observe o Request URL em Headers.

vlab

FIELD GUIDE

OWASP Top 10:2025 — nosso recorte

01	A01 — Broken Access Control → IDOR
02	A05 — Injection → SQL Injection e XSS em alguns contextos
03	A07 — Authentication Failures → sessão e credenciais
04	A02 — Security Misconfiguration → configurações ruins

São 10 categorias no documento oficial; aqui estudaremos algumas na prática.

INTERACTIVE // THINK

SE O BROWSER PODE MUDAR A REQUEST...

O QUE O SERVIDOR PRECISA VALIDAR?

Pense em autenticação, autorização, formato e regras de negócio.

vlab

MODULE 02 / ACCESS GRANTED

02

BURP SUITE

Intercepte, modifique e reenviar.

01101001 10010110 11100010
01011011 00110100 11001010
10101101 01001011 11101001

ATTACK SURFACE

Burp Community: nossos 4 painéis

PROXY

Intercept

Segura requisições antes do servidor.

HTTP HISTORY

Histórico

Mostra tráfego observado.

REPEATER

Laboratório

Edita e reenvia uma request.

BROWSER

Pronto para uso

Navegador pré-configurado com proxy.

REQUEST PIPELINE

O Burp entra no meio

Proxy é observação + edição de mensagens. Não é “hack” automático.



STEP BY STEP / FOLLOW ALONG

Primeiro acesso no Burp

01	**Abra Burp Community**	Iniciar → Burp Suite Community Edition
02	**Temporary project**	Clique Next na tela de criação do projeto
03	**Use Burp defaults**	Clique Start Burp e aguarde
04	**Proxy → Intercept**	Clique Open browser
05	**Navegue só no laboratório**	http://localhost:8088/login

Nos próximos slides, configure a interceptação e o Repeater com detalhes.

EXECUTE / PASSO A PASSO

Burp 01 — iniciar o projeto

01	**Abra Burp Suite Community**	Inicie o programa pelo menu Iniciar do Windows.
02	**Na tela de projeto**	Selecione Temporary project.
03	**Clique Next**	Em seguida selecione Use Burp defaults.
04	**Clique Start Burp**	Aguarde aparecer a interface principal.

OK **A aba Proxy está disponível no Burp.**

SE FALHAR: Se solicitar licença: confira se instalou Community Edition.

vlab

EXECUTE / PASSO A PASSO

Burp 02 — confirme a porta do proxy

01	**Abra Settings**	Ícone de engrenagem no Burp.
02	**Vá a Tools → Proxy**	Procure a seção Proxy listeners.
03	**Confirme listener Running**	Endereço 127.0.0.1 e porta 8080.
04	**Não altere a porta do site**	A NEON SHOP continua em localhost:8088.

OK **Proxy = 8080; aplicação = 8088. São dois serviços distintos.**

SE FALHAR: Se o listener não estiver Running, verifique conflito na porta 8080.

CONTRASTE

Burp 03 — não confunda as duas portas

BURP PROXY (INTERMEDIÁRIO)

- 127.0.0.1:8080
- Recebe requisições do navegador
- Permite inspecionar/alterar

NEON SHOP (APLICAÇÃO)

- 127.0.0.1:8088
- Responde as requisições HTTP
- Roda no Docker local

vlab

EXECUTE / PASSO A PASSO

Burp 04 — abra o navegador CORRETO

01	**Feche Settings**	Volte à janela principal do Burp.
02	**Selecione Proxy**	Abra a subaba Intercept.
03	**Clique Open browser**	Uma nova janela Chromium deve abrir.
04	**Nessa janela**	Digite http://localhost:8088/login

OK **O NEON SHOP abre no navegador embutido do Burp.**

SE FALHAR: Se você navegou no Chrome normal, a request pode NÃO passar pelo Burp.

vlab

EXECUTE / PASSO A PASSO

Burp 05 — histórico sem travar a página

01	**Abra Proxy → Intercept**	Confira o botão “Intercept is off”.
02	**Se estiver ON**	Clique para mudar para “Intercept is off”.
03	**No navegador do Burp**	Faça login em /login.
04	**Abra Proxy → HTTP history**	Encontre chamadas ao host localhost:8088.

OK **As requests aparecem no histórico mesmo com Intercept OFF.**

SE FALHAR: Se não aparecer nada, veja se está no navegador do Burp e se o filtro do History oculta localhost.

vlab

EXECUTE / PASSO A PASSO

Burp 06 — encontre a request de login

01	**HTTP history**	Use a coluna Host para localizar localhost:8088.
02	**Procure a linha**	POST /api/login (geralmente status 200).
03	**Clique na linha**	Abra as seções Request e Response.
04	**Leia o body**	JSON com email e password fictícios.

OK **Você identifica o HTTP real que o JavaScript enviou.**

SE FALHAR: Se houver ruído, use o filtro do histórico; não apague a sessão.

vlab

EXECUTE / PASSO A PASSO

Burp 07 — ligue a interceptação

01	**Burp: Proxy → Intercept**	Clique no botão “Intercept is off”.
02	**Verifique o texto**	Ele deve passar a “Intercept is on”.
03	**Navegador do Burp**	Abra /orders e clique CONSULTAR.
04	**Volte ao Burp**	A request aparecerá pausada na área Intercept.

OK **A página pode parecer “carregando”: isso é esperado com Intercept ON.**

SE FALHAR: Se não parar: recarregue e confirme o navegador do Burp.

vlab

EXECUTE / PASSO A PASSO

Burp 08 — encaminhe ou descarte

01	**Leia a primeira linha**	GET /api/orders/... HTTP/1.1.
02	**Clique Forward**	Libera a request para a API local.
03	**Observe o navegador**	A página termina de carregar.
04	**Clique Intercept is on**	Deixe OFF para não bloquear outros cliques.

OK **Você sabe destravar uma request pausada.**

SE FALHAR: Se o browser continuar carregando, podem existir outras requests: Forward novamente ou deixe OFF.

vlab

EXECUTE / PASSO A PASSO

Burp 09 — edite ANTES de encaminhar

01	**Volte a Intercept ON**	No Burp: Proxy → Intercept.
02	**No navegador**	Consulte o pedido 1001 em /orders.
03	**Na request pausada**	Troque /api/orders/1001 por /api/orders/1002.
04	**Clique Forward**	O servidor recebe o caminho MODIFICADO.

OK **A página recebe a resposta referente a 1002.**

SE FALHAR: Se a request for /api/me ou /api/health, faça Forward até capturar /api/orders/1001.

EXECUTE / PASSO A PASSO

Burp 10 — envie ao Repeater

01	**Intercept OFF**	Deixe o navegador funcionando normalmente.
02	**Abra Proxy → HTTP history**	Selecione GET /api/orders/1001.
03	**Clique com botão direito**	Escolha Send to Repeater.
04	**Abra a aba Repeater**	A request estará disponível para edição.

OK **O Repeater mostra a request à esquerda e espaço para resposta à direita.**

SE FALHAR: Se enviou outra request, volte ao HTTP history e selecione o caminho correto.

vlab

EXECUTE / PASSO A PASSO

Burp 11 — qual texto você modifica?

01	**No Repeater**	Abra a aba Raw/Request (texto editável).
02	**Leia a primeira linha**	GET /api/orders/1001 HTTP/1.1.
03	**Altere SÓ o caminho**	Substitua 1001 por 1002, sem mexer no Cookie.
04	**Clique Send**	A resposta aparecerá no painel da direita.

OK **Compare o JSON antes/depois e mantenha a sessão de Alex.**

SE FALHAR: Se o status for 401, faça login novamente e capture uma request com cookie válido.

vlab

EXECUTE / PASSO A PASSO

Burp 12 — saiba interpretar a Response

01	**Leia o status**	200 = atendida; 401 = sem autenticação; 404 = ausente.
02	**Abra a aba Response**	Identifique o objeto JSON retornado.
03	**Procure owner_id**	O servidor retornou aluno-A ou aluno-B?
04	**Registre a evidência**	Anote método, URL, status e recurso.

OK **Você consegue explicar o que mudou sem depender da interface da loja.**

SE FALHAR: Se aparecer 404, confira o identificador digitado.

vlab

EXECUTE / PASSO A PASSO

Burp 13 — se não aparecer tráfego

01	**Confira a aba**	Proxy → HTTP history (não Repeater).
02	**Confirme o browser**	Abra o navegador por Proxy → Intercept → Open browser.
03	**Confira listener**	Settings → Tools → Proxy → 127.0.0.1:8080 Running.
04	**Desative filtros**	No HTTP history, remova filtros e recarregue /orders.

OK **GET /api/orders passa a aparecer no histórico.**

SE FALHAR: Se o navegador travar, deixe Intercept OFF; se o serviço estiver parado, use docker compose ps.

EXECUTE / PASSO A PASSO

Burp 14 — feche a interceptação ao terminar

01	**Proxy → Intercept**	Clique até ler “Intercept is off”.
02	**Volte ao browser**	Cheque se a navegação funciona.
03	**Preserve o histórico**	Não precisa limpar HTTP history.
04	**Continue no Repeater**	Próximos exercícios partirão desse fluxo.

OK **Você pode navegar livremente e ainda analisar requests.**

SE FALHAR: Se uma request ficar presa, clique Forward ou deixe Intercept OFF.

vlab

STEP BY STEP / FOLLOW ALONG

Burp — configure em menos de 2 minutos

01	**Abra Burp Community**	Temporary project → Use Burp defaults.
02	**Proxy → Intercept**	Deixe Intercept OFF para navegar livremente.
03	**Clique Open browser**	Use o navegador já configurado com proxy.
04	**Acesse o laboratório**	http://localhost:8088/login
05	**Faça login como Aluno A**	Depois abra Proxy → HTTP history.

Se OFF, o histórico continua capturando as requests.

STEP BY STEP / FOLLOW ALONG

Burp — transformar um clique em experimento

01	**Abra /orders**	Clique VER no pedido 1001.
02	**Abra HTTP history**	Procure GET /api/orders/1001.
03	**Botão direito**	Escolha Send to Repeater.
04	**Troque apenas o ID**	1001 → 1002; mantenha a sessão.
05	**Clique Send**	Compare status code e JSON da resposta.

O navegador do Burp é o caminho recomendado: evita configurar proxy/CA manualmente.

VISUAL MODEL

HTTP HISTORY: a trilha dos cliques

GET /api/orders → 200

Liste método, caminho, status e tamanho. Procure parâmetros interessantes.

CONTRASTE

Intercept OFF x ON

INTERCEPT OFF

- Fluxo passa normalmente
- Histórico continua disponível
- Ideal para navegar

INTERCEPT ON

- Request pode ficar pausada
- Edite antes de encaminhar
- Forward ou Drop

vlab

RAW HTTP / INSPECT

Interceptando uma request

REQUEST / RESPONSE

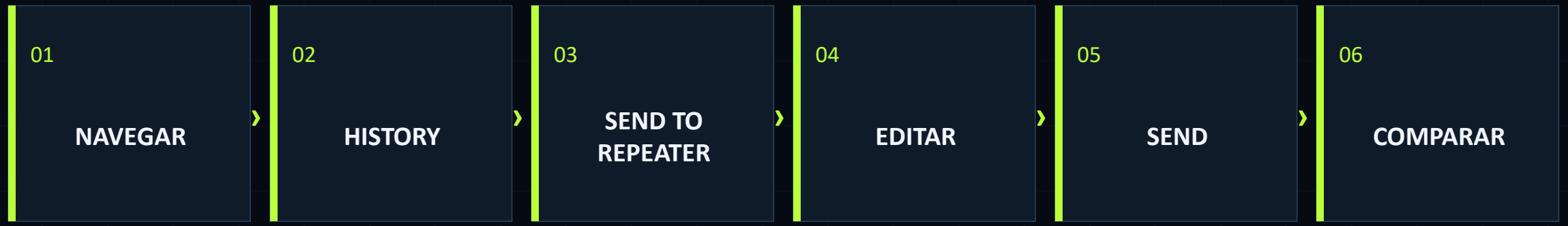
```
GET /api/orders/1001 HTTP/1.1
Host: localhost:8088
Cookie: session=aluno-A
Accept: application/json
```

Qual parte depende do usuário? Qual parte identifica o recurso?

REQUEST PIPELINE

Do histórico para o Repeater

O mesmo request, diferentes entradas, respostas diferentes.



vlab

RAW HTTP / INSPECT

Repeater: altere UMA variável

REQUEST / RESPONSE

```
GET /api/orders/1001 HTTP/1.1
Cookie: session=aluno-A

--- MUDE APENAS ---
GET /api/orders/1002 HTTP/1.1
```

Experimento controlado: mantenha a sessão e altere o ID.

INTERACTIVE // THINK

O FRONTEND ENVIOU 1001.

E SE EU ENVIAR 1002?

Quem deve impedir o acesso indevido?

HANDS-ON / LAB CHALLENGE

MISSÃO 02 — Seu primeiro Repeater

12 MIN

NÍVEL 02

Reenvie uma request e compare 2 respostas.

- Ache uma request em HTTP History.
- Use Send to Repeater.
- Altere somente um parâmetro.

Entregável: antes/depois e sua hipótese.

KEY TAKEAWAY

O CLIENTE FAZ PEDIDOS. O SERVIDOR IMPÕE AS REGRAS. REGRAS.

Tudo que aparece no navegador pode ser inspecionado e alterado.

03

IDOR / BROKEN ACCESS CONTROL

Uma troca de ID pode expor dados de terceiros.

01101001 10010110 11100010
01011011 00110100 11001010
10101101 01001011 11101001

SCENARIO / INCIDENT

O caso do pedido 1001

ALUNO A

PEDIDO #1001

“E se eu pedir o #1002?”

Mesmo autenticado, ele não deveria ver o pedido de outra pessoa.

RAW HTTP / INSPECT

Pedido legítimo

REQUEST / RESPONSE

```
GET /api/orders/1001 HTTP/1.1
Cookie: session=aluno-A

HTTP/1.1 200 OK
{"order_id":1001,"owner":"aluno-A"}
```

Autenticado e autorizado: mesmo dono.

RAW HTTP / INSPECT

Pedido de outra pessoa

REQUEST / RESPONSE

```
GET /api/orders/1002 HTTP/1.1
Cookie: session=aluno-A

HTTP/1.1 200 OK  // VULNERÁVEL
{"order_id":1002,"owner":"aluno-B"}
```

O resultado esperado é negar acesso — não retornar dados de aluno-B.

VISUAL MODEL

IDOR: o erro não é adivinhar o ID

FALTA CHECAR O DONO

IDs numéricos, UUIDs ou slugs não substituem uma política de autorização.

ROOT CAUSE / FIX

A falha no backend

VULNERABLE / BEFORE

```
id := c.Param("id")
db.Get(&order,
  "SELECT * FROM orders WHERE id=$1", id)
```

FIX / INSIGHT

A consulta não considera quem está autenticado.

ROOT CAUSE / FIX

A correção: restringir por identidade

VULNERABLE / BEFORE

```
SELECT * FROM orders
WHERE id = $1
```

FIX / INSIGHT

```
SELECT * FROM orders
WHERE id = $1 AND owner_id = $2
```

CONTRASTE

404 ou 403?

403 // FORBIDDEN

- Identidade conhecida
- Sem permissão para acessar
- Pode revelar existência do recurso

404 // NOT FOUND

- Recurso não encontrado ou ocultado
- Pode reduzir enumeração
- Consistência > código mágico

vlab

ATTACK SURFACE

IDOR aparece em muitos lugares

/orders/42

Pedidos

Consulta de dados.

/files/42

Arquivos

Download privado.

/users/42

Perfil

Leitura ou edição.

/companies/42

Multi-tenant

Fronteira entre organizações.

DEFENSE CHECKLIST

Checklist de autorização por objeto

Autorização é uma regra do negócio, não da interface.

- [ ] O usuário foi autenticado?
- [ ] O objeto pertence a ele ou à sua organização?
- [ ] A ação específica é permitida?
- [ ] O filtro é aplicado no servidor?
- [ ] Existe teste para acesso cruzado?
- [ ] O mesmo vale para PUT/PATCH/DELETE?

vlab

HANDS-ON / LAB CHALLENGE

MISSÃO 03 — Acesso cruzado

15 MIN

NÍVEL 02

Com a conta A, tente acessar um recurso pertencente à conta B.

- Encontre o pedido da conta A.
- Mantenha cookie de A.
- Troque somente o ID para o pedido B.
- Registre status e corpo da resposta.

Entregável: evidência do comportamento e correção sugerida.

vlab

EXECUTE / PASSO A PASSO

IDOR 01 — estabeleça o caso normal

01	**Burp browser: /login**	Entre como aluno@lab.local / exemplo123.
02	**Abra /orders**	Veja a lista de pedidos do Alex.
03	**Localize pedido 1001**	Clique em VER ou informe 1001 e CONSULTAR.
04	**Leia a resposta**	owner_id deve ser aluno-A.

OK **Pedido 1001 é permitido para a conta A.**

SE FALHAR: Se for Beatriz ou Admin, clique SAIR e entre como Aluno A.

vlab

EXECUTE / PASSO A PASSO

IDOR 02 — capture o pedido autorizado

01	**Burp em Intercept OFF**	Abra Proxy → HTTP history.
02	**Na loja**	Abra /orders e consulte 1001.
03	**No histórico**	Selecione GET /api/orders/1001.
04	**Send to Repeater**	Botão direito na linha.

OK **A request no Repeater contém sessão válida do Aluno A.**

SE FALHAR: Se encontrar uma request /api/orders sem número, consulte o pedido individual.

vlab

EXECUTE / PASSO A PASSO

IDOR 03 — teste o acesso cruzado

01	**No Repeater → Request**	Mude apenas 1001 para 1002.
02	**Não altere Cookie**	Você deve continuar autenticado como Alex.
03	**Clique Send**	Observe status e body da Response.
04	**Encontre owner_id**	Verifique se aparece aluno-B.

OK **Vulnerável: HTTP 200 e pedido 1002 de outra conta.**

SE FALHAR: Se 401, capture uma nova request após login; se 404, pode estar no modo secure.

vlab

EXECUTE / PASSO A PASSO

IDOR 04 — compreenda a evidência

01	**Leia o campo evidence**	Ele aparece no pedido 1002 em modo vulnerável.
02	**Copie o valor**	Formato NEON{IDOR_...}.
03	**Abra /ctf**	No navegador do Burp ainda logado.
04	**Cole → VALIDAR**	A missão IDOR soma 150 pontos.

OK **O scoreboard registra a prova do acesso não autorizado.**

SE FALHAR: Não digite uma flag de outro PC: cada ambiente usa uma semente distinta.

vlab

EXECUTE / PASSO A PASSO

IDOR 05 — encontre o bug no código

01	**Abra VS Code / Zed**	Abra a pasta clonada neon-shop-infoeste.
02	**Entre em app/server.py**	Use Ctrl+F e procure “VULNERABLE IDOR”.
03	**Leia a consulta**	WHERE id=? busca por pedido sem conferir owner_id.
04	**Leia a opção SECURE**	WHERE id=? AND owner_id=? é a correção.

OK **Você identifica o controle ausente no backend.**

SE FALHAR: Não edite a versão usada no CTF; o modo seguro será ativado via Compose no fim.

EXECUTE / PASSO A PASSO

IDOR 06 — prove a mitigação

01	**Após ativar mode secure**	Faça logout e login novamente como Alex.
02	**Abra /orders**	Consulte 1001 e confirme acesso legítimo.
03	**Repita /api/orders/1002**	Via formulário ou Repeater.
04	**Compare a Response**	A resposta agora deve ser 404.

OK **Uma conta não consegue ler o pedido pertencente à outra.**

SE FALHAR: Se ainda retornar 200, confirme /api/health: mode deve ser secure.

vlab

RAW HTTP / INSPECT

IDOR — request exata do NEON SHOP

REQUEST / RESPONSE

```
GET /api/orders/1001 HTTP/1.1
Host: localhost:8088
Cookie: session=aluno-A

# No Repeater: altere SOMENTE /1001 para /1002
```

Somente laboratório. O cookie fictício é criado no login; envie a request pela sessão do Aluno A.

KEY TAKEAWAY

AUTENTICADO NÃO QUER DIZER AUTORIZADO.

Valide o sujeito, o objeto e a ação em cada operação.

MODULE 04 / ACCESS GRANTED

04

SQL INJECTION

Quando um valor vira parte do comando SQL.

01101001 10010110 11100010
01011011 00110100 11001010
10101101 01001011 11101001

VISUAL MODEL

A aplicação deveria enviar DADOS

INPUT ≠ SQL

A vulnerabilidade nasce quando o código mistura um valor controlável com a estrutura da query.

ROOT CAUSE / FIX

Uma query que parece inocente

VULNERABLE / BEFORE

```
query := "SELECT * FROM users WHERE email='"
  + email + "'"
```

FIX / INSIGHT

O valor do usuário foi concatenado no comando SQL.

RAW HTTP / INSPECT

Payload de laboratório

REQUEST / RESPONSE

```
GET /api/search?q=' OR '1'='1 HTTP/1.1
Host: localhost:8088

# Exemplo didático em ambiente local
```

Exemplo simples: o objetivo é observar alteração indevida da lógica da busca.

RAW HTTP / INSPECT

SQLi — formato correto da request

REQUEST / RESPONSE

```
GET /api/search?q=%27%20OR%20%271%27%3D%271 HTTP/1.1
Host: localhost:8088

# O valor decodificado de q seria:
# ' OR '1'='1
```

Compare com uma pesquisa normal antes de alterar a requisição.

REQUEST PIPELINE

A entrada virou lógica

O problema é estrutural: dado interpretado como código.



vlab

CONTRASTE

Duas formas de montar SQL

INSEGURO

- String concatenada
- Comando e dados misturados
- Entrada pode mudar lógica

SEGURO

- Query parametrizada
- Parâmetros separados
- Banco distingue código e dado

ROOT CAUSE / FIX

Prepared statement na prática

VULNERABLE / BEFORE

```
q := "... WHERE email = '" + email + "'"
```

FIX / INSIGHT

```
q := "SELECT id FROM users WHERE
email=$1"
err := db.Get(&userID, q, email)
```

DEFENSE CHECKLIST

Defesas contra SQL Injection

Escapar strings manualmente não é a primeira linha de defesa.

- [x] Usar queries parametrizadas
- [x] Evitar SQL dinâmico com concatenação
- [x] Allowlist para colunas e ordenações dinâmicas
- [x] Privilégios mínimos no banco
- [x] Não expor detalhes SQL nas respostas
- [x] Testar entradas inválidas e limites

HANDS-ON / LAB CHALLENGE

MISSÃO 04 — A busca virou SQL

18 MIN

NÍVEL 03

Compare o comportamento normal da busca com uma entrada malformada no laboratório. laboratório.

- Capture a request de busca.
- Altere q no Repeater.
- Observe status, mensagens e resultados.
- Identifique o ponto de concatenação.

Entregável: prova + query parametrizada.

vlab

EXECUTE / PASSO A PASSO

SQLi 01 — observe a busca normal

01	**No navegador do Burp**	Abra http://localhost:8088/search.
02	**Digite mouse**	Clique no botão BUSCAR.
03	**Observe resultados**	Devem aparecer produtos comuns.
04	**Abra HTTP history**	Procure GET /api/search?q=...

OK **Resposta 200 e lista de produtos sem o registro 99.**

SE FALHAR: Se não aparecer a chamada, filtre pela palavra search no HTTP history.

EXECUTE / PASSO A PASSO

SQLi 02 — prepare a request no Repeater

01	**Selecione GET /api/search**	No Proxy → HTTP history.
02	**Botão direito**	Escolha Send to Repeater.
03	**Abra Repeater**	Localize o parâmetro q= na primeira linha.
04	**Clique Send sem modificar**	Guarde uma resposta de referência.

OK **A resposta normal NÃO contém LAB-INTERNAL-ARCHIVED.**

SE FALHAR: Se houver erro de SQL, restaure a request original e envie novamente.

RAW HTTP / INSPECT

SQLi 03 — request modificada, pronta para testar

REQUEST / RESPONSE

```
GET /api/search?q=%25%27%20OR%201%3D1%20--%20 HTTP/1.1
Host: localhost:8088

# q decodificado = %' OR 1=1 --
```

No Repeater, substitua apenas o valor de q=. Preserve os headers reais capturados.

vlab

EXECUTE / PASSO A PASSO

SQLi 04 — compare as duas respostas

01	**No Repeater**	Envie a request com q alterado.
02	**Leia o status**	Esperado: HTTP 200.
03	**Busque no JSON**	Procure LAB-INTERNAL-ARCHIVED ou id:99.
04	**Observe description**	Ela inclui evidence: NEON{SQLI_...}.

OK **Produto oculto aparece por manipulação da condição SQL.**

SE FALHAR: Se houver HTTP 400, copie exatamente o parâmetro codificado do slide anterior.

EXECUTE / PASSO A PASSO

SQLi 05 — registre a prova

01	**Na Response**	Copie somente o valor de evidence na descrição.
02	**Abra a loja /ctf**	Use a sessão do Aluno A.
03	**Cole a flag SQLI**	No campo FLAG ENCONTRADA.
04	**Clique VALIDAR**	Observe mais 200 pontos.

OK **O CTF contabiliza a SQL Injection.**

SE FALHAR: A flag pode estar dentro do texto de description, não como campo evidence separado.

vlab

EXECUTE / PASSO A PASSO

SQLi 06 — localize a causa no servidor

01	**Abra app/server.py**	No editor escolhido.
02	**Ctrl+F → VULNERABLE SQLI**	Localize a concatenação do parâmetro q.
03	**Entenda a falha**	O texto do usuário entra na string do SQL.
04	**Compare SECURE**	O banco usa placeholders ? e parâmetros separados.

OK **Você explica a diferença entre concatenar e parametrizar.**

SE FALHAR: Se editar código local, o container não muda automaticamente; não é necessário editar para este exercício.

vlab

EXECUTE / PASSO A PASSO

SQLi 07 — reexecute no modo seguro

01	**Ative secure pelo Compose**	Veja o slide de ativação no encerramento.
02	**Abra /search**	Faça uma busca normal: ela continua funcionando.
03	**No Repeater**	Reenvie o parâmetro q malicioso.
04	**Compare o resultado**	Produto 99 não deve aparecer.

OK **Sem exposição do registro oculto; parâmetros passam como dados.**

SE FALHAR: Se o 99 ainda aparecer, verifique /api/health e a reconstrução do container.

INTERACTIVE // THINK

A QUERY FOI PARAMETRIZADA.

A SEGURANÇA ACABOU?

Pense em permissões do banco, logs e outras consultas.

KEY TAKEAWAY

SEPARAR CÓDIGO E DADOS É A REGRA DE OURO.

Correção estrutural > filtro improvisado de palavras.

vlab

MODULE 05 / ACCESS GRANTED

05

CROSS-SITE SCRIPTING

Dados exibidos como código executável.

01101001 10010110 11100010
01011011 00110100 11001010
10101101 01001011 11101001

vlab

SCENARIO / INCIDENT

O mural de comentários

ALUNO A

NOVO COMENTÁRIO

“E se a mensagem contiver HTML?”

O navegador deveria exibir texto. Uma implementação insegura pode interpretar marcação.

VISUAL MODEL

XSS: texto vira execução

DADO → HTML / JS

O risco depende de ONDE e COMO a entrada é inserida na página.

vlab

RAW HTTP / INSPECT

Payload visual de laboratório

REQUEST / RESPONSE

```
POST /api/comments HTTP/1.1
Content-Type: application/json

{"text":"<img src=x onerror=\"alert(1)\">"}
```

Exemplo benigno e executável no DOM vulnerável deste laboratório; só execute em localhost.

STEP BY STEP / FOLLOW ALONG

XSS — exemplos que funcionam neste laboratório

01	**Abra /comments**	Entre como Aluno A.
02	**Teste a formatação HTML**	NEON TEST
03	**Teste um evento inofensivo**	<img src=x onerror="alert('INFOESTE')">
04	**Compare o resultado**	HTML interpretado x texto exibido.

Uma tag <script> inserida por innerHTML normalmente NÃO executa: use o teste de evento acima.

ATTACK SURFACE

Três contextos de XSS

REFLECTED

Refletido

Entrada volta na resposta.

STORED

Armazenado

Conteúdo persiste e aparece depois.

DOM-BASED

No cliente

JavaScript insere o conteúdo em contexto contexto inseguro.

ROOT CAUSE / FIX

O perigo no DOM

VULNERABLE / BEFORE

```
box.innerHTML = comment.text;
```

FIX / INSIGHT

```
box.textContent = comment.text;
```

ROOT CAUSE / FIX

React ajuda, mas não faz milagre

VULNERABLE / BEFORE

```
<div dangerouslySetInnerHTML={{
 __html: comment.text
}} />
```

FIX / INSIGHT

```
<div>{comment.text}</div>
```

CONTRASTE

Encoding x sanitização

OUTPUT ENCODING

- Exibir como texto
- Depende do contexto
- Padrão para texto não confiável

HTML SANITIZATION

- Permitir HTML restrito
- Remover elementos perigosos
- Usar biblioteca confiável

vlab

DEFENSE CHECKLIST

Defesa em profundidade para XSS

CSP e HttpOnly ajudam, mas não substituem o tratamento correto.

- [x] Preferir APIs que tratam entrada como texto
- [x] Evitar sinks perigosos no DOM
- [x] Usar output encoding contextual
- [x] Usar CSP como defesa adicional
- [x] Sanitizar HTML permitido com biblioteca mantida
- [x] Proteger sessão com cookies HttpOnly quando adequado

vlab

HANDS-ON / LAB CHALLENGE

MISSÃO 05 — Comentários perigosos

18 MIN

NÍVEL 03

Identifique a inserção insegura e faça o conteúdo aparecer sem executar código.

- Publique texto com caracteres HTML.
- Observe como a aplicação renderiza.
- Localize o sink no frontend.
- Troque pela API segura de texto.

Entregável: antes/depois e motivo técnico.

vlab

EXECUTE / PASSO A PASSO

XSS 01 — comece com texto normal

01	**Entre como Alex**	Abra http://localhost:8088/comments.
02	**Na caixa SUA MENSAGEM**	Digite: Teste INFOESTE 2026
03	**Clique PUBLICAR COMENTÁRIO**	Veja o texto no mural abaixo.
04	**No Burp history**	Encontre POST /api/comments e GET /api/comments.

OK **A API armazena um comentário e o browser o renderiza.**

SE FALHAR: Se receber 401, volte a /login; o POST exige sessão.

EXECUTE / PASSO A PASSO

XSS 02 — descubra a interpretação de HTML

01	**Na caixa de comentário**	Digite TESTE NEON
02	**Clique PUBLICAR**	Localize sua mensagem mais recente.
03	**Observe o visual**	“TESTE NEON” aparece em negrito.
04	**Pense no risco**	O servidor guardou texto; o browser interpretou HTML.

OK **O elemento HTML é interpretado no modo vulnerable.**

SE FALHAR: Se aparecer literalmente , confira se o modo secure está desligado.

RAW HTTP / INSPECT

XSS 03 — payload didático da prova

REQUEST / RESPONSE

```
<img src=x onerror="neonProofXSS()">
```

Cole EXATAMENTE na caixa de comentários. Não há envio para servidor externo.

vlab

EXECUTE / PASSO A PASSO

XSS 04 — publique e observe a prova

01	**Ainda em /comments**	Cole o HTML do slide anterior.
02	**Clique PUBLICAR COMENTÁRIO**	O item aparecerá no feed.
03	**Observe XSS / CONCEITO**	A caixa Aguardando prova benigna muda.
04	**Procure evidence**	Ela contém NEON{XSS_...}.

OK **Função local executada e flag XSS exibida na página.**

SE FALHAR: Se o campo não mudar, recarregue /comments e verifique o HTML colado sem espaços extras.

vlab

EXECUTE / PASSO A PASSO

XSS 05 — identifique a linha insegura

01	**Abra app/web/app.js**	No editor.
02	**Procure “VULNERABLE DOM SINK” SINK”**	Ctrl+F.
03	**Observe innerHTML**	Insere c.text no HTML do mural.
04	**Compare com secure**	esc(c.text) trata caracteres como texto.

OK **Você localiza um sink perigoso do lado do navegador.**

SE FALHAR: O exemplo do slide é do Neon Shop; não confunda sanitização, encoding e validação no backend.

EXECUTE / PASSO A PASSO

XSS 06 — faça a comparação com a defesa

01	**Ative o modo secure**	Reabra /comments.
02	**Olhe comentário antigo**	A tag <img ...> aparece como texto literal.
03	**Publique um novo payload**	Reutilize o texto do slide XSS 03.
04	**Observe a tela**	Não deve executar a função de prova.

OK **A entrada aparece como texto; não há execução no DOM.**

SE FALHAR: Se houver execução, confirme via health se mode realmente é secure.

KEY TAKEAWAY

EXIBIR TEXTO NÃO É EXECUTAR TEXTO.

A defesa depende do contexto de saída.

MODULE 06 / ACCESS GRANTED

06

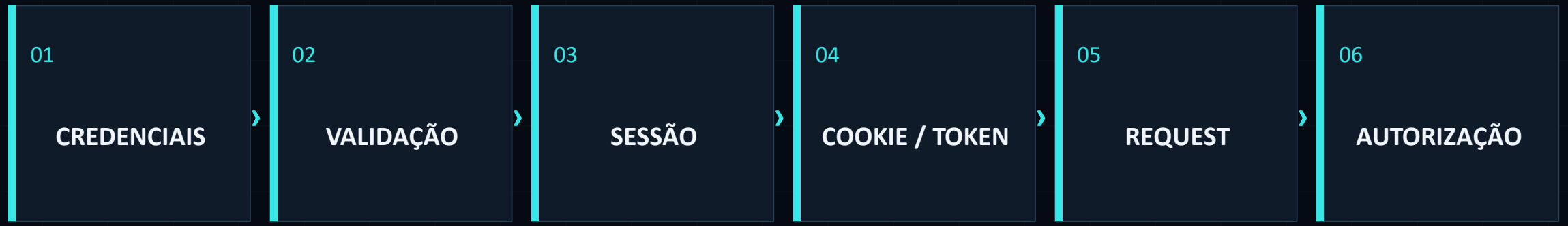
AUTH + SESSÕES

Identidade válida ainda precisa de validações corretas.

01101001 10010110 11100010
01011011 00110100 11001010
10101101 01001011 11101001

REQUEST PIPELINE

Um login tem mais passos que parece



vlab

CONTRASTE

Cookie de sessão x JWT

SESSÃO

- Identificador opaco
- Estado normalmente no servidor
- Revogação pode ser direta

JWT

- Claims no payload
- Assinatura precisa ser validada
- Expiração e revogação exigem estratégia

VISUAL MODEL

JWT é legível. Isso não é falha.

HEADER . PAYLOAD . SIGNATURE

Base64url codifica; a assinatura verifica integridade. Decodificar não é validar.

DEFENSE CHECKLIST

Checklist de validação de tokens

- [x] Assinatura criptográfica verificada
- [x] Algoritmos aceitos explicitamente
- [x] Issuer (iss) validado
- [x] Audience (aud) validada
- [x] Expiration (exp) aplicada
- [x] Chaves e rotação controladas
- [x] Autorização continua no servidor

vlab

STEP BY STEP / FOLLOW ALONG

JWT — veja o token antes de testá-lo

01	**Abra /login**	Entre com aluno@lab.local / exemplo123.
02	**Abra /admin**	Leia a mensagem de permissão.
03	**Clique DECODIFICAR JWT**	Observe o payload e a role.
04	**Tente Solicitar relatório**	Observe a resposta de API: 403.

Decodificar um JWT NÃO equivale a verificar sua assinatura.

ATTACK SURFACE

Onde guardar sessão no browser?

HttpOnly

Cookie

Protege contra leitura direta por JavaScript.

Secure

Cookie

Transmissão restrita a HTTPS.

SameSite

Cookie

Ajuda a mitigar certos cenários CSRF.

localStorage

Storage

Acessível ao JavaScript da mesma origem.

vlab

INTERACTIVE // THINK

VOCÊ FEZ LOGOUT.

O TOKEN ANTIGO AINDA FUNCIONA?

O que muda entre sessão revogável e token que só expira?

FIELD GUIDE

Senhas: três regras inegociáveis

01	Nunca armazenar senha em texto puro.
02	Usar password hashing apropriado (Argon2id ou bcrypt conforme o sistema).
03	Nunca registrar credenciais ou tokens em logs.

Hash de senha não é “criptografar e descriptografar”.

vlab

HANDS-ON / LAB CHALLENGE

MISSÃO 06 — Auth não é autorização

12 MIN

NÍVEL 03

Investigue uma regra de acesso que depende apenas do frontend ou de uma claim confiada confiada indevidamente.

- Compare conta comum e conta admin de teste.
- Observe identidade, sessão e rotas.
- Explique qual decisão deve estar no servidor.

Entregável: regra corrigida em uma frase.

vlab

EXECUTE / PASSO A PASSO

JWT 01 — compare conta e permissão

01	**No browser Burp**	Login como aluno@lab.local.
02	**Abra http://localhost:8088/admin**	Veja TOKEN JWT — EDITÁVEL.
03	**Clique LER PAYLOAD**	Role deve ser student.
04	**Clique SOLICITAR RELATÓRIO**	Deve retornar erro de autorização 403.

OK **Usuário comum NÃO possui permissão administrativa por padrão.**

SE FALHAR: Se o token estiver vazio, saia e faça login novamente na mesma janela.

EXECUTE / PASSO A PASSO

JWT 02 — abra o console no lugar certo

01	**Mantenha /admin aberto**	No mesmo navegador que fez login.
02	**Pressione F12**	Abra a aba Console.
03	**Veja onde escrever**	Clique na linha de entrada após o prompt >.
04	**Atenção ao aviso do browser**	Se o DevTools bloquear colar, digite o snippet ou siga o instrutor. instrutor.

OK **Console executa JavaScript somente nesta aba do laboratório.**

SE FALHAR: Não cole código aleatório da Internet; o snippet seguinte opera apenas no token fictício local.

RAW HTTP / INSPECT

JWT 03 — copie e cole no Console (parte A)

REQUEST / RESPONSE

```
const t = document.querySelector("#jwt-value").value;
const [h, p, s] = t.split(".");
const data = JSON.parse(atob(p.replace(/-/g,"+").replace(/_/g,"/")));
data.role = "admin";
```

No DevTools → Console da página /admin. Ainda não envie nada.

RAW HTTP / INSPECT

JWT 04 — Console (parte B): montar token

REQUEST / RESPONSE

```
const newP = btoa(JSON.stringify(data))
  .replace(/\+/g,"-").replace(/\//g,"_")
  .replace(/=+$/,"");
const forged = `${h}.${newP}.${s}`;
console.log(forged);
```

Execute após o trecho da parte A; copie o token exibido no console.

vlab

EXECUTE / PASSO A PASSO

JWT 05 — onde colar o token alterado?

01	**Volte à aba /admin**	Feche o Console ou deixe-o à direita.
02	**Localize TOKEN JWT — EDITÁVEL**	Caixa grande no painel esquerdo.
03	**Substitua tudo**	Cole o token alterado impresso no Console.
04	**Clique LER PAYLOAD**	Confira que role passou a admin.

OK **O payload decodificado mostra role: admin.**

SE FALHAR: Se o token quebrar, repita partes A+B e copie a linha inteira até a assinatura.

vlab

EXECUTE / PASSO A PASSO

JWT 06 — resultado do backend vulnerável

01	**Com token alterado**	Clique SOLICITAR RELATÓRIO.
02	**Leia a resposta**	A solicitação deve receber status 200.
03	**Localize evidence**	NEON{AUTH_...} aparece no JSON.
04	**Cole em /ctf**	Valide a flag AUTH (250 pontos).

OK **Conta student ganhou acesso porque a assinatura não foi verificada.**

SE FALHAR: Se continuar 403, role não mudou; se 401, o token ficou malformado.

vlab

EXECUTE / PASSO A PASSO

JWT 07 — compare com o código-fonte

01	**Abra app/server.py**	Localize GET /api/admin/reports.
02	**Procure decode_jwt**	Compare verify=SECURE.
03	**Observe vulnerable**	Claims podem ser confiadas sem assinatura.
04	**Observe secure**	Valida assinatura e confere role real no banco.

OK **O bug é confiar em claims alteráveis sem validar sua integridade.**

SE FALHAR: Ler o conteúdo de um token não prova que ele foi emitido pelo servidor.

vlab

MODULE 07 / ACCESS GRANTED

07

FALHAS DO DIA A DIA

O bug inseguro que nasce em decisões comuns.

01101001 10010110 11100010
01011011 00110100 11001010
10101101 01001011 11101001

SCENARIO / INCIDENT

“É só um upload de avatar...”

USUÁRIO

ARQUIVO ENVIADO

“O servidor confia na extensão?”

Upload exige validação, armazenamento seguro, limites e política de acesso.

vlab

DEFENSE CHECKLIST

Checklist de upload seguro

- [x] Allowlist de formatos necessários
- [x] Validar conteúdo e não apenas extensão/MIME
- [x] Limitar tamanho e quantidade
- [x] Gerar nome de arquivo pelo servidor
- [x] Guardar fora de diretório executável
- [x] Servir com headers adequados e controle de acesso

TERMINAL / POWERSHELL

Upload — criar um PNG falso no Windows

SESSION // LOCAL ONLY

```
Set-Content -Path .\fake.png -Value "isto nao e um png"
Get-Item .\fake.png
```

Depois abra /upload, selecione fake.png, MIME image/png e compare os modos.

EXECUTE / PASSO A PASSO

Upload 01 — crie um arquivo falso no Windows

01	**Abra PowerShell**	Pode ser na pasta do repositório.
02	**Execute o comando**	Set-Content -Path .\fake.png -Value "nao sou PNG"
03	**Confira o arquivo**	Get-Item .\fake.png
04	**Entenda o teste**	Extensão .png, conteúdo real de texto.

OK **fake.png existe, mas NÃO possui assinatura PNG.**

SE FALHAR: Se não encontrar, use Get-Location para descobrir onde o PowerShell está salvando.

vlab

EXECUTE / PASSO A PASSO

Upload 02 — envie pelo formulário

01	**No navegador logado**	Abra http://localhost:8088/upload.
02	**Clique Escolher arquivo**	Selecione fake.png criado no PowerShell.
03	**MIME DECLARADO**	Mantenha image/png.
04	**Clique ENVIAR METADADOS**	Observe o resultado no painel.

OK **Modo vulnerable aceita o arquivo e devolve HTTP 201.**

SE FALHAR: Se não aceitar, confira a extensão .png e limite de 100 KB.

vlab

EXECUTE / PASSO A PASSO

Upload 03 — ache a request no Burp

01	**Abra HTTP history**	No Burp Proxy.
02	**Localize POST /api/upload**	Selecione essa chamada.
03	**Observe o body JSON**	Procure filename, mime e content_b64.
04	**Compare**	Extensão/MIME podem mentir sobre os bytes.

OK **A API recebeu metadados e conteúdo em Base64.**

SE FALHAR: Se não aparece no histórico, faça o upload pelo navegador do Burp.

EXECUTE / PASSO A PASSO

Upload 04 — bloqueio no modo seguro

01	**Ative secure**	Depois do CTF.
02	**Login novamente**	Abra /upload.
03	**Selecione o mesmo fake.png**	MIME image/png.
04	**Clique ENVIAR METADADOS**	Compare com a execução anterior.

OK **Modo secure rejeita o conteúdo falso com HTTP 415.**

SE FALHAR: Se for aceito, confirme mode: secure via /api/health.

vlab

CONTRASTE

Validação no frontend x backend

FRONTEND

- Melhora experiência
- Mostra mensagens amigáveis
- Pode ser ignorado ou alterado

BACKEND

- É a decisão final
- Valida regra e permissão
- Não confia no cliente

vlab

ATTACK SURFACE

Quatro erros que vazam segredos

LOGS

Tokens expostos

Credenciais e dados sensíveis.

CONFIG

Debug em produção

Stack traces e detalhes internos.

CÓDIGO

Segredo hardcoded

Chaves no repositório.

DEPENDÊNCIAS

Pacote vulnerável

Atualizações e provenance.

VISUAL MODEL

Segurança também é observabilidade

SEM LOG = SEM EVIDÊNCIA

Registre ações relevantes, sem armazenar segredos ou dados desnecessários.

KEY TAKEAWAY

NÃO BASTA BLOQUEAR O ATAQUE.

Você precisa identificar, registrar e investigar comportamentos suspeitos.

MODULE 08 / ACCESS GRANTED

08

OWASP ZAP

Do raciocínio manual ao apoio automatizado.

01101001 10010110 11100010
01011011 00110100 11001010
10101101 01001011 11101001

ATTACK SURFACE

O que o ZAP pode ajudar a enxergar

CRAWL

Descoberta

Navegação e rotas visíveis.

PASSIVE

Análise passiva

Observa mensagens sem alterar o alvo.

ACTIVE

Teste ativo

Envia requests de teste ao alvo autorizado.

ALERTS

Achados

Organiza evidências para triagem.

vlab

TERMINAL / POWERSHELL

ZAP Quick Start — laboratório local

SESSION // LOCAL ONLY

1 Abrir OWASP ZAP
2 Quick Start
3 Automated Scan
4 URL: http://localhost:8088
5 Attack (SOMENTE NO LAB)

Verifique escopo e impacto antes de qualquer active scan.

vlab

STEP BY STEP / FOLLOW ALONG

ZAP — roteiro de varredura autorizada

01	**Abra OWASP ZAP**	Quick Start → Automated Scan.
02	**Informe a URL**	http://localhost:8088
03	**Confirme o escopo**	Somente loopback; nunca IP de colegas.
04	**Inicie a análise**	Attack: espere a varredura do laboratório.
05	**Abra Alerts**	Escolha 1 achado e valide manualmente.

Não espere que o scanner detecte todas as falhas de autorização.

EXECUTE / PASSO A PASSO

ZAP 01 — não confunda ZAP e Burp

01	**No Burp**	Deixe Intercept OFF e feche seu navegador embutido.
02	**Abra OWASP ZAP**	Pelo menu Iniciar do Windows 11.
03	**Na pergunta sobre sessão**	Escolha a opção temporária, se preferir.
04	**Aguarde Quick Start**	A guia deve aparecer no workspace.

OK **ZAP iniciou separadamente do Burp.**

SE FALHAR: Se houver conflito de proxy, use o navegador iniciado pelo próprio ZAP.

EXECUTE / PASSO A PASSO

ZAP 02 — primeiro faça exploração manual

01	**Quick Start**	Clique Manual Explore.
02	**URL to explore**	Digite http://localhost:8088.
03	**Select your browser**	Escolha Chrome ou Firefox disponível.
04	**Clique Launch Browser**	Navegue só no NEON SHOP.

OK **O ZAP registra URLs visitadas em Sites/History.**

SE FALHAR: Se não capturar, confira a janela aberta pelo ZAP, não o browser normal.

vlab

EXECUTE / PASSO A PASSO

ZAP 03 — explore páginas relevantes

01	**Na janela do ZAP**	Faça login no laboratório fictício.
02	**Clique pelas páginas**	Orders, Search, Comments, Admin e Upload.
03	**Volte ao ZAP**	Abra árvore Sites e o painel History.
04	**Inspecione uma mensagem**	Veja request, response e headers.

OK **Algumas URLs descobertas aparecem na árvore Sites.**

SE FALHAR: ZAP não entende automaticamente todas as ações JavaScript de uma SPA.

EXECUTE / PASSO A PASSO

ZAP 04 — Automated Scan

01	**Abra a aba Quick Start**	Clique Automated Scan.
02	**URL to attack**	Digite http://localhost:8088.
03	**Confirme escopo**	URL aponta para a máquina LOCAL, porta 8088.
04	**Clique Attack**	Somente depois da orientação do instrutor.

OK **O ZAP começa a descobrir URLs e testá-las.**

SE FALHAR: Não rode sobre IP da faculdade, sistemas externos ou máquinas de colegas.

vlab

EXECUTE / PASSO A PASSO

ZAP 05 — acompanhe o andamento

01	**Observe abas inferiores**	Spider, Active Scan e Progress podem aparecer.
02	**Aguarde o processo**	Não interrompa no meio sem necessidade.
03	**Abra Sites**	Veja páginas que foram descobertas.
04	**Abra Alerts**	Observe os achados gerados.

OK **URLs descobertas e, se houver, alertas para analisar.**

SE FALHAR: Se um alerta não surgir, isso não significa automaticamente que não há falhas.

vlab

EXECUTE / PASSO A PASSO

ZAP 06 — valide um alerta sem adivinhar

01	**Selecione um alerta**	Na aba Alerts.
02	**Leia o Risk e Description**	Qual é a hipótese do scanner?
03	**Veja a evidence/request**	Que entrada levou ao resultado?
04	**Reproduza no Burp**	Se viável, use Repeater no alvo local.

OK **Você consegue classificar o achado como confirmado, incerto ou falso positivo.**

SE FALHAR: Não confunda risco calculado com evidência suficiente de exploração.

EXECUTE / PASSO A PASSO

ZAP 07 — IDOR pode escapar ao scanner

01	**Lembre do IDOR**	Pedidos 1001 e 1002 pertencem a usuários distintos.
02	**Compare a análise do ZAP**	Ele sabe quem é dono de cada pedido?
03	**Volte ao Burp Repeater**	Faça a comparação entre identidades.
04	**Conclua**	Automação ajuda, mas autorização exige contexto.

OK **Você entende limites da detecção automática de lógica de negócio.**

SE FALHAR: Não atribua a scanners resultados que não foram observados.

vlab

CONTRASTE

Scanner e analista

AUTOMAÇÃO

- Cobertura repetível
- Padrões técnicos conhecidos
- Muitos alertas para validar

ANÁLISE MANUAL

- Contexto de negócio
- Permissões entre usuários
- Explorações dependentes de fluxo

HANDS-ON / LAB CHALLENGE

MISSÃO 07 — Validar alertas

12 MIN

NÍVEL 03

Faça um scan somente no localhost e selecione um alerta para verificar manualmente. manualmente.

- Defina escopo local.
- Execute análise de laboratório.
- Abra a evidência de um alerta.
- Confirme ou descarte com raciocínio.

Entregável: alerta, evidência, conclusão.

KEY TAKEAWAY

SCANNER ENCONTRA SINAIS. GENTE ENCONTRA CONTEXTO. CONTEXTO.

Não confunda alertas com vulnerabilidades confirmadas.

vlab

MODULE 10 / ACCESS GRANTED

01101001

10010110

11100010

01011011

00110100

9

DEFESA EM PROFUNDIDADE

Do ataque pontual à postura de segurança — web e nuvem.

vlab

OWASP Top 10 — o mapa completo

A01 — **Broken Access Control**
IDOR, path traversal, força de rotas

A02 — **Cryptographic Failures**
dados sensíveis sem cifra ou cifra fraca

A03 — **Injection**
SQLi, XSS, command, LDAP

A04 — **Insecure Design**
falha de modelagem, não de bug

A05 — **Security Misconfiguration**
defaults, debug, headers ausentes

A06 — **Vulnerable Components**
dependências desatualizadas

A07 — **Auth Failures**
sessão, credenciais, JWT

A08 — **Integrity Failures**
CI/CD e update sem verificação

A09 — **Logging & Monitoring**
sem detecção não há resposta

A10 — **SSRF**
servidor forçado a requisitar interno

As 5 do lab (A01, A03, A07) são a ponta do iceberg — as 10 compõem a avaliação real.

vlab

Modelagem de ameaças — STRIDE

Pergunte “o que pode dar errado?” antes de escrever a primeira linha.

S **Spoofing**

fingir ser outro

→ **Autenticação forte**

T **Tampering**

alterar dados/código

→ **Integridade, assinatura**

R **Repudiation**

negar ter feito

→ **Logs e auditoria**

I **Information Disclosure**

vazar dados

→ **Cifra, least privilege**

D **Denial of Service**

derrubar o serviço

→ **Rate limit, quotas**

E **Elevation of Privilege**

virar admin

→ **AuthZ no servidor**

NO LAB O JWT forjado foi Spoofing + Elevation of Privilege; o IDOR, Information Disclosure.

vlab

Superfície de ataque e fronteira de confiança

Tudo que o cliente controla

- URL, parâmetros, headers, cookies
- Body, uploads, campos ocultos
- Ordem e repetição de requests
- Qualquer valor — mesmo os “fixos” do front

A fronteira de confiança

- Onde dado não-confiável vira decisão
- Sempre no servidor — nunca no browser
- Cada endpoint é uma porta
- Minimize o que é exposto (menos rotas, menos risco)

PRINCÍPIO

Nunca confie na entrada. Valide tipo, formato, tamanho e faixa; aplique allowlist, não blocklist. O frontend melhora a experiência, mas a decisão experiência, mas a decisão de segurança é sempre do backend.

NO LAB Todas as 5 falhas nasceram de dado do cliente cruzando a fronteira sem validação do servidor.

vlab

AuthN moderna — provar quem você é

Fatores (MFA)

- Algo que sabe (senha)
- Algo que tem (TOTP, passkey)
- Algo que é (biometria)
- Senha sozinha não basta

OAuth2 / OIDC

- Delegação sem compartilhar senha
- OIDC = identidade sobre OAuth2
- Access token x ID token
- SSO corporativo

Sessão x Token

- Cookie: opaco, revogável
- JWT: stateless, expira
- Guardar hash, nunca a senha
- Argon2id / bcrypt

ARMADILHA COMUM

Decodificar um JWT não é validá-lo. O payload é Base64url (legível por qualquer um); só a verificação da assinatura prova que o token foi emitido o token foi emitido pelo servidor.

NO LAB O bypass de admin funcionou porque o backend lia as claims sem conferir a assinatura HS256.

vlab

AuthZ — o que você pode fazer

RBAC x ABAC

- RBAC: papéis (admin, user)
- ABAC: atributos (dono, org, horário)
- Combine: papel + propriedade
- Decisão centralizada e testável

Regras de ouro

- Deny-by-default
- Verifique sujeito + objeto + ação
- Autorização por objeto (ownership)
- Vale para GET e também PUT/PATCH/DELETE

AUTHN ≠ AUTHZ

Estar autenticado diz quem você é; autorização decide se você pode tocar aquele recurso. **Esconder um botão no front não é controle de acesso. controle de acesso.**

NO LAB O IDOR corrige-se com autorização por objeto: WHERE id=? AND owner_id=? — no servidor.

vlab

Sessão e cookies seguros

HttpOnly
JS não lê o cookie — reduz impacto de XSS

Secure
só trafega sobre HTTPS

SameSite
Lax/Strict — mitiga CSRF

Rotação
novo ID a cada login; invalide no logout

Expiração
TTL curto + refresh controlado

Fixation
nunca aceite ID de sessão vindo do cliente

NO LAB localStorage guarda o JWT — e é lido por JS: por isso XSS que rouba token é tão grave.

vlab

Criptografia aplicada

Senhas

- Hash lento + salt
- Argon2id (preferido) ou bcrypt
- Hash ≠ criptografia reversível
- Nunca MD5/SHA simples

Em trânsito

- TLS 1.2+ em tudo
- HSTS para forçar HTTPS
- Sem HTTP em produção
- Certificados válidos e rotacionados

Em repouso

- Cifra de disco/DB
- Gestão de chaves (KMS)
- Separe chave de dado
- Rotação de chaves

REGRA

Não invente cripto. Use bibliotecas mantidas e padrões revisados; o erro quase nunca está no algoritmo, e sim no uso (chave fraca, sem salt, dado fraca, sem salt, dado exposto no log).

vlab

Validação de entrada e output encoding

Entrada (contra injeção)

- Queries parametrizadas (nunca concatenar)
- Allowlist de formato, tipo e tamanho
- Valide no servidor, sempre
- Separar código de dados = regra de ouro

Saída (contra XSS)

- Output encoding conforme o contexto
- HTML, atributo, JS e URL são contextos distintos
- textContent em vez de innerHTML
- Sanitizar HTML permitido com lib confiável

CONTEXTO IMPORTA

O mesmo dado é seguro num lugar e perigoso em outro. Codifique na saída, de acordo com onde o valor será inserido — e trate toda entrada trate toda entrada como não-confiável.

NO LAB SQLi = entrada virou código; XSS = dado virou HTML. Mesma raiz: código e dado misturados.

vlab

Headers de segurança e CSP

Content-Security-Policy
restringe origens de script/estilo — mitiga XSS

Strict-Transport-Security
força HTTPS (HSTS)

X-Content-Type-Options
nosniff — impede MIME sniffing

X-Frame-Options
anti-clickjacking (frame-ancestors)

Referrer-Policy
controla vazamento de referrer

CORS
libere origens com critério, não *

NO LAB O ZAP apontou CSP, X-Frame e nosniff ausentes — no modo seguro o backend passa a enviá-los.

vlab

Segredos e cadeia de suprimentos

Segredos

- Nunca no código ou no Git
- Cofre: Vault, Secrets Manager
- Variáveis de ambiente / injeção em runtime
- Rotação e escopo mínimo

Dependências

- SCA: varre libs vulneráveis
- SBOM: inventário do que você usa
- Fixe versões (pin) + provenance
- Atualize com disciplina

SUPPLY CHAIN

Seu código é tão seguro quanto suas dependências. Um pacote comprometido ou uma chave vazada no repositório derruba todo o resto — todo o resto — ataques recentes miram exatamente a cadeia.

vlab

Shift-left — segurança desde o código

01
DESIGN
Threat modeling, requisitos de segurança segurança

›

02
CÓDIGO
SAST, revisão de pares, pares, linters de segurança

›

03
BUILD
SCA de dependências, assinatura de artefatos

›

04
TEST
DAST, IAST, testes de abuso

›

05
DEPLOY
Gates no CI/CD, IaC scan, secrets check

›

06
RUN
Monitoramento, WAF, resposta a incidentes

Corrigir no design custa ~1x; em produção, ~100x. Segurança é atividade contínua, não etapa final.

vlab

DEEP DIVE / AS 10 CATEGORIAS

01000001
01010000
01010000
01010011
01000101

10

OWASP TOP 10:2025 em detalhe

Para cada categoria: o que é, como se explora e um exemplo prático. Edição 2025 — lançada em nov/2025.

vlab

A01 Broken Access Control

Nº1 · absorve SSRF

O QUE É	Usuário faz ou lê o que não deveria — acessar dado de outro, usar rota de admin, ou forçar o servidor a buscar um recurso interno (SSRF, agora dentro desta categoria).
COMO EXPLORAR	Trocar IDs na URL ou no token (IDOR), acessar /admin direto, adulterar cookie ou campo "role", apontar a aplicação para aplicação para endpoints internos.
EXEMPLO PRÁTICO	/api/orders/1001 → 1002 devolve o pedido de outro cliente; SSRF para http://169.254.169.254 vaza credenciais da credenciais da nuvem.

NO LAB foi o IDOR (trocar 1001→1002) e o acesso indevido a /admin.

vlab

A02 Security Misconfiguration

↑ 5º → 2º

O QUE É	Configuração insegura: contas e senhas padrão, serviços expostos, headers de segurança ausentes, erros verbosos, armazenamento público.
COMO EXPLORAR	Testar credenciais default, achar diretório listável, ler stack trace que vaza a arquitetura, abrir um bucket sem sem autenticação.
EXEMPLO PRÁTICO	Painel com admin/admin; /actuator ou .git expostos; bucket S3 público com dados de clientes.

NO LAB o ZAP apontou CSP, X-Frame e nosniff ausentes — tudo misconfiguration.

vlab

A03 Software Supply Chain Failures

NOVA · 2025

O QUE É	Categoria nova. Comprometer o que constrói e distribui o software: uma dependência maliciosa, um update não assinado, um pipeline ou registry infiltrado.
COMO EXPLORAR	Typosquatting e dependency confusion (publicar pacote com nome parecido), roubar o token de um mantenedor e mantenedor e republicar uma versão envenenada.
EXEMPLO PRÁTICO	Worm Shai-Hulud no npm (2025), que se autorreplica; SolarWinds (2020): um update legítimo entregue com backdoor. com backdoor.

vlab

A04 Cryptographic Failures

↓ 2º → 4º

O QUE É	Dado sensível sem cifra ou com cifra fraca; chave mal gerida; TLS ausente. O problema quase nunca é o algoritmo — é o uso errado.
COMO EXPLORAR	Interceptar tráfego sem HTTPS, quebrar hash fraco (MD5/SHA1 sem salt), achar segredo hardcoded no repositório. repositório.
EXEMPLO PRÁTICO	Senhas guardadas em MD5; API em HTTP puro; JWT HS256 com segredo fraco, quebrável com hashcat.

NO LAB o token do lab é HS256 — segredo fraco = token forjável.

vlab

A05 Injection

↓ 3º → 5º

O QUE É	Entrada não-confiável interpretada como comando ou código: SQL, HTML/JavaScript (XSS), comandos de SO, LDAP. Raiz: misturar código e dado.
COMO EXPLORAR	' OR '1'='1 numa busca (SQLi); <img src=x onerror=...> num campo (XSS); ; cat /etc/passwd concatenado num comando. num comando.
EXEMPLO PRÁTICO	Busca que concatena o texto direto na query SQL; comentário renderizado como HTML no mural.
NO LAB	foram o /search (SQLi) e o /comments (XSS armazenado).

vlab

A06 Insecure Design

↓ 4º → 6º

O QUE É	Falha na arquitetura, não um bug pontual: faltou prever o controle já no desenho do sistema. Não se corrige com um patch, se corrige repensando o fluxo.
COMO EXPLORAR	Abusar de um fluxo mal modelado: "esqueci a senha" fraco, ausência de rate limit, regra de negócio que pode ser pode ser contornada.
EXEMPLO PRÁTICO	Recuperação de senha que revela se o e-mail existe; checkout que aceita preço negativo; transferência sem limite. sem limite.

vlab

A07 Authentication Failures

estável

O QUE É	Falhas ao provar identidade: sessão, credenciais e tokens. "Decodificar não é validar" — o servidor precisa verificar a assinatura.
COMO EXPLORAR	Brute force e credential stuffing, sessão previsível, JWT aceito sem verificar assinatura, logout que não invalida o token. invalida o token.
EXEMPLO PRÁTICO	Login sem limite de tentativas; token aceito mesmo depois de adulterado.

NO LAB foi o JWT forjado (role=admin) aceito em /api/admin/reports.

vlab

A08 Software and Data Integrity Failures

estável

O QUE É	Confiar em código ou dado sem checar integridade: deserialização insegura, update sem assinatura, pipeline de CI/CD comprometido.
COMO EXPLORAR	Insecure deserialization levando a execução remota (RCE); empurrar um artefato malicioso; adulterar um objeto objeto serializado.
EXEMPLO PRÁTICO	Deserialização Java/PHP → RCE; auto-update sem verificação de assinatura instala código do atacante.

vlab

A09 Security Logging & Alerting Failures

renomeada

O QUE É	Falta de log e de alerta: o ataque não é detectado nem pode ser investigado. Em 2025 "Monitoring" virou "Alerting" — o foco é reagir, não só registrar.
COMO EXPLORAR	Não é explorada diretamente; o atacante se aproveita da cegueira para persistir sem ser visto e apagar rastros. rastros.
EXEMPLO PRÁTICO	Brute force sem nenhum alerta disparado; invasão descoberta meses depois — média de 241 dias para detectar e conter. detectar e conter.

vlab

A10 Mishandling of Exceptional Conditions

NOVA · 2025

O QUE É	Categoria nova (substitui o SSRF na 10ª posição). A aplicação reage mal ao inesperado: exceção não tratada que vaza stack, race condition, ou "fail open" (libera acesso quando dá erro).
COMO EXPLORAR	Forçar um erro para vazar informação ou cair num estado inseguro; TOCTOU; corrida em saldo ou estoque. estoque.
EXEMPLO PRÁTICO	Erro 500 com stack trace completo; checagem de permissão que, ao falhar, concede acesso em vez de negar. negar.

vlab

Parece igual, mas não é

A07 · AUTHENTICATION FAILURES

AUTENTICAÇÃO — AuthN

“quem é você?”

Falha em provar a identidade: senha fraca, brute force, sessão previsível, token aceito sem verificar a assinatura.

A01 · BROKEN ACCESS CONTROL

AUTORIZAÇÃO — AuthZ

“o que você pode?”

Já está autenticado, mas acessa além da permissão: ler o pedido de outro outro (IDOR), abrir rota de admin.

A07 falha na PORTA — não devia nem ter entrado. **A01 falha DENTRO** — entrou, mas foi aonde não podia.

Outras confusões comuns:

A02 Misconfiguration ≠ A06 Insecure Design

O controle existe mas está mal configurado (default, header ausente) × o controle nunca foi previsto no desenho.

A05 Injection ≠ A10 Exceptional Conditions

Sua entrada vira comando/código × a app reage mal a um erro (vaza stack, “fail stack, “fail open”).

vlab

2021 → 2025

2 categorias novas

Software Supply Chain Failures (A03) e Mishandling of Exceptional Conditions (A10)

SSRF foi absorvido

deixou de ser categoria própria — agora faz parte de Broken Access Control (A01)

Misconfiguration subiu

de 5º para 2º — reflexo de cloud e deploy contínuo sem checagem contínua contínua

Injection caiu

de 3º para 5º — é das falhas mais testadas hoje, mas segue comum

A09 renomeada

"Monitoring" → "Alerting": não basta logar, tem que detectar e reagir

Broken Access Control

segue em 1º pela 4ª edição seguida

No nosso lab: IDOR = A01 · SQLi e XSS = A05 · JWT = A07. As três campeãs de aplicações reais.

vlab

MODULE 11 / ACCESS GRANTED

01101001
10010110
11100010
01011011
00110100

11

SEGURANÇA EM CLOUD

A aplicação não vive sozinha: ela roda em infraestrutura hospedada.

vlab

Modelo de responsabilidade compartilhada

O provedor cuida de

- Segurança DA nuvem
- Hardware, datacenter, rede física
- Hypervisor e serviços gerenciados
- Disponibilidade da plataforma

Você cuida de

- Segurança NA nuvem
- IAM, dados, configuração
- Patches do que você gerencia
- Código da aplicação

A maioria dos incidentes em nuvem é erro de configuração do cliente — não falha do provedor. O bucket público, a role permissiva e o segredo permissiva e o segredo no repo são responsabilidade sua.

vlab

IAM na nuvem — menor privilégio

Least privilege
conceda só o necessário, por tarefa

Roles, não chaves
credenciais temporárias > chaves longas

Sem root no dia a dia
conta raiz trancada + MFA

Políticas explícitas
deny-by-default, escopo por recurso

Federação / SSO
identidade central, sem usuários locais

Auditoria de acesso
revise permissões periodicamente

NO LAB Mesma lição do AuthZ do app: privilégio mínimo — agora aplicado a quem acessa a infra.

vlab

Rede — segmentar e desconfiar

Isolamento

- VPC / subnets privadas
- Security groups restritivos
- Nada de 0.0.0.0/0 sem motivo
- Bastion / sem SSH público

Zero Trust

- Nunca confie, sempre verifique
- Autentique serviço-a-serviço
- Microssegmentação
- Perímetro não basta

Borda

- WAF à frente do app
- Rate limiting / anti-DDoS
- TLS terminando na borda
- CDN com origem protegida

NO LAB WAF teria sinalizado o ' OR '1'='1 — mas é camada extra, não substitui a query parametrizada.

vlab

Dados na nuvem — cifra e exposição

Proteção

- Cifra em repouso (KMS) e em trânsito
- Buckets privados por padrão
- Backup testado + versionamento
- Classifique dado sensível (PII)

O clássico

- Bucket S3 público por engano
- Snapshot/DB exposto à internet
- Log com dado sensível
- URL assinada sem expiração

O VAZAMENTO MAIS COMUM

Armazenamento mal configurado é a causa nº1 de vazamento em nuvem. Private-by-default, cifra sempre, e acesso só via identidade — nunca identidade — nunca “link público” como controle.

vlab

Configuração e segredos na nuvem

Segredos gerenciados

- Secrets Manager / Vault / KMS
- Rotação automática
- Injeção em runtime, não em imagem
- Escopo por serviço

Infra as Code segura

- Versione a infra (Terraform)
- Scan de IaC (tfsec, checkov)
- Revisão + aprovação como código
- Drift detection

CONFIG É CÓDIGO

Misconfiguration é A05 do OWASP e a falha mais comum em nuvem. Tratar infra como código permite revisar, versionar e escanear a escanear a configuração antes de ela ir ao ar.

vlab

Observabilidade e detecção

Registrar

- Logs de acesso e de app
- Audit trail (quem fez o quê)
- Sem segredo/PII no log
- Centralize (SIEM)

Detectar

- Alertas de anomalia
- Trilha de auditoria da nuvem
- Métricas + traces
- Monitoração contínua

Responder

- Plano de resposta a incidentes
- Runbooks e ensaios
- Contenção e evidência
- Post-mortem sem culpados

Sem log não há evidência; sem detecção não há resposta. Bloquear o ataque é metade — você precisa saber que ele aconteceu e conseguir aconteceu e conseguir investigar.

vlab

Contêineres — superfície cloud-native

Imagem mínima
base slim/distroless, menos CVEs

Non-root
nunca rode como root (USER)

Read-only FS
filesystem imutável + tmpfs

Scan de imagem
Trivy/Grab na pipeline

Sem segredo na imagem
injete em runtime

Cap drop / seccomp
menor privilégio no kernel

NO LAB O Dockerfile do lab já aplica isso: python-slim, USER 65534, read_only, cap_drop ALL.

vlab

Compliance e frameworks

LGPD
proteção de dados pessoais (BR)

ISO 27001
gestão de segurança da informação

SOC 2
controles para provedores de serviço

OWASP ASVS
níveis de verificação de AppSec

CIS Benchmarks
hardening por tecnologia

NIST CSF
identificar→proteger→detectar→responder

Framework não é burocracia: é um checklist testável do que “seguro o suficiente” significa para o seu contexto.

vlab

CLOSING / SÍNTESE

ATACAR ENSINA. DEFENDER PROTEGE.

Defense in depth

Camadas: nenhuma defesa sozinha basta

Servidor decide

A fronteira de confiança é o backend

Menor privilégio

Do WHERE owner_id ao IAM da nuvem

Seguro por padrão

Private, deny-by-default, config como código

Observar sempre

Sem log não há evidência nem resposta

Contínuo

Shift-left: segurança em todo o ciclo

Estudar a seguir: OWASP ASVS · Top 10 · Cheat Sheet Series · CIS Benchmarks · PortSwigger Web Security Academy

vlab

TERMINAL / POWERSHELL

SOS — o site não está abrindo

SESSION // LOCAL ONLY

```
docker compose ps
docker compose logs --tail=80 neon-shop
curl.exe -i http://127.0.0.1:8088/api/health
Test-NetConnection 127.0.0.1 -Port 8088
```

Confirme Docker Desktop aberto e que a porta 8088 não está sendo usada por outro processo.

STEP BY STEP / FOLLOW ALONG

SOS — problemas comuns da aula

502 / não abre	Verifique Docker Desktop e docker compose compose ps.
Burp sem tráfego	Use Proxy → Open browser; Intercept OFF. OFF.
Request travada	Intercept ON exige Forward ou OFF.
Sem permissão permissão em em /admin	Essa é a validação normal para Aluno A.
Flags antigas sumiram	Evite docker compose down -v.

O arquivo docs/TROUBLESHOOTING.md do Git detalha os casos.

EXECUTE / PASSO A PASSO

SOS 01 — localhost:8088 não abre

01	**Verifique Docker Desktop**	O engine precisa estar iniciado.
02	**Rode docker compose ps**	Procure Up (healthy) em neon-shop.
03	**Rode curl.exe /api/health**	Veja se recebe HTTP 200.
04	**Veja logs**	docker compose logs --tail=80 neon-shop.

OK **Diagnóstico separa erro do browser de falha na API.**

SE FALHAR: Se a porta 8088 estiver ocupada, não exponha em 0.0.0.0; peça assistência.

vlab

EXECUTE / PASSO A PASSO

SOS 02 — o browser do Burp não navega

01	**Abra Burp → Proxy**	Vá a Intercept.
02	**Desative interceptação**	O botão deve dizer Intercept is off.
03	**Abra o browser interno**	Clique Open browser.
04	**Digite a URL local**	http://localhost:8088/login.

OK **Página carrega e o HTTP history registra requests.**

SE FALHAR: Se o proxy não funciona, verifique Settings → Tools → Proxy → listeners Running.

EXECUTE / PASSO A PASSO

SOS 03 — response 401 ou 403

01	401	Credenciais/cookie/token inválidos ou ausentes.
02	403	Usuário autenticado sem autorização.
03	No navegador Burp	Faça logout/login para renovar a sessão.
04	No Repeater	Reenvie request nova capturada com Cookie correto.

OK **O erro tem interpretação técnica; nem todo bloqueio é bug.**

SE FALHAR: Não invente header Authorization: selecione sempre uma request real capturada.

vlab

EXECUTE / PASSO A PASSO

SOS 04 — flag não é aceita

01	**Confira formato**	O valor deve iniciar em NEON{ e terminar em }.
02	**Copie sem espaços**	Não inclua aspas ou vírgulas da resposta JSON.
03	**Confira o usuário**	Permaneça logado na mesma máquina.
04	**Não apague o volume**	Isso altera a semente e invalida flags anteriores.

OK **CTF aceita evidência do próprio container.**

SE FALHAR: Se a flag antiga não funcionar após reset, encontre a evidência novamente.

TERMINAL / POWERSHELL

Encerramento — pare sem apagar progresso

SESSION // LOCAL ONLY

```
docker compose down

# APENAS se quiser APAGAR dados e flags:
# docker compose down -v
```

Nunca use down -v durante a avaliação; o volume local guarda comentários e pontuação.

STEP BY STEP / FOLLOW ALONG

Checklist de saída: você consegue?

01	**Subir o laboratório**	docker compose up -d --build
02	**Inspecionar requests**	DevTools e Burp HTTP history.
03	**Reenviar uma request**	Burp Repeater; altere uma variável.
04	**Validar a correção**	Modo secure e nova requisição.
05	**Parar sem perder dados**	docker compose down

Todos os comandos estão também em docs/ALUNO.md, disponível no repositório Git.

MISSION COMPLETE

//

THE BEST EXPLOIT IS A FIXED BUG.

Observe. Teste com autorização. Entenda a causa. Corrija. Repita.

vlab

MINI CTF / ROUND 2 · ACESSO LIVRE

01010110
01000001
01010101
01001100
01010100

VAULT

Banco Digital — seu segundo CTF

Mesmas 5 técnicas, alvo novo. Agora sem roteiro — vocês conduzem.

vlab

Como funciona

O desafio

- 90 minutos, vocês sozinhos (sem passo a passo)
- Mesmas 5 vulnerabilidades — app totalmente novo
- Speedrun: placar ao vivo em /ctf, 900 pontos
- Primeiro a fechar as 5 vence

Entregável por flag

- Request + Response (print do Burp)
- Qual era a vulnerabilidade (a causa)
- Como se corrige no servidor
- Não abram o código pra achar a flag — investiguem o HTTP

FLAGS POR PC

Cada máquina gera suas próprias flags no formato VAULT{...}. A flag do colega não vale na sua tela — o que importa é o caminho até não a string.

vlab

Suba o alvo

01 **Clonar**

```
git clone https://github.com/HenriqueOtsuka/
vault-bank-infoeste.git
cd vault-bank-infoeste
```

02 **Subir no Docker**

```
docker compose up -d --build
```

03 **Abrir e logar**

http://127.0.0.1:8099/login
cliente@vault.local / exemplo123

Confira: `curl -i http://127.0.0.1:8099/api/health` → mode: vulnerable. Façam tudo logados como Ana.

vlab

Mapa da VAULT

RECON	/robots.txt → /api/internal/diag	+100
IDOR	/api/transactions/<id>	+150
SQLi	/api/search?q=	+200
XSS	/mensagens (mural)	+200
AUTH	/admin → /api/admin/reports	+250

Total: **900 pontos** · Burp no proxy 127.0.0.1:8080, alvo 127.0.0.1:8099

vlab

LARGADA

Boa caçada. ⏱ 90 min

- **Logados como Ana**
  cliente@vault.local · exemplo123
- **Investiguem o HTTP**
  não abram o código pra achar flag
- **Flag por PC**
  colem em /ctf no formato VAULT{...}
- **Placar ao vivo**
  primeiro a fechar 900 vence

Repo: github.com/SEU-USUARIO/vault-bank-infoeste